Principles of Pipelining

Basic Computer Architecture

Outline

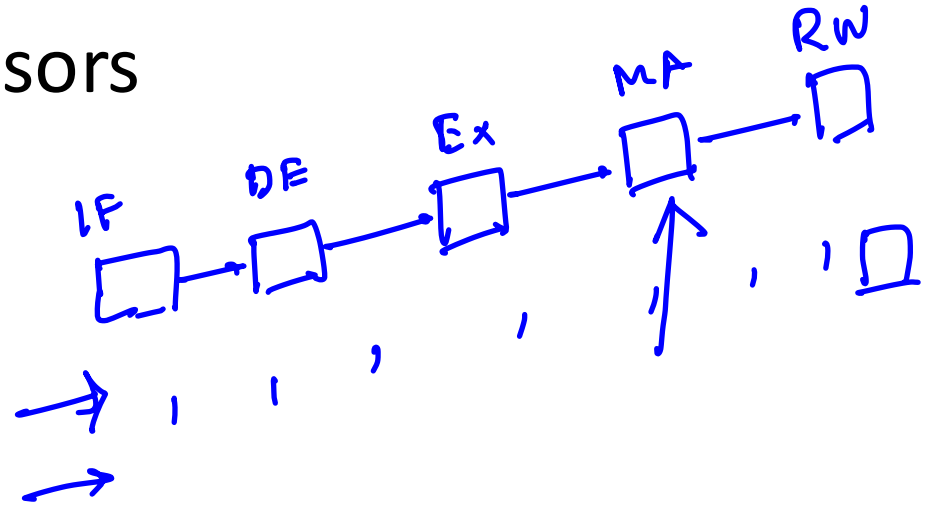
* Overview of Pipelining
* A Pipelined Data Path
* Pipeline Hazards
* Pipeline with Interlocks
* Forwarding
* Performance Metrics
* Interrupts/ Exceptions

Up till now ….

- We have designed a processor that can execute all the SimpleRisc Instructions
- We have look at two styles :
  - With a hardwired control unit
  - Microprogrammed control unit
    - Microprogrammed data path
    - Microassembly Language
    - Microinstructions

Designing Efficient Processors

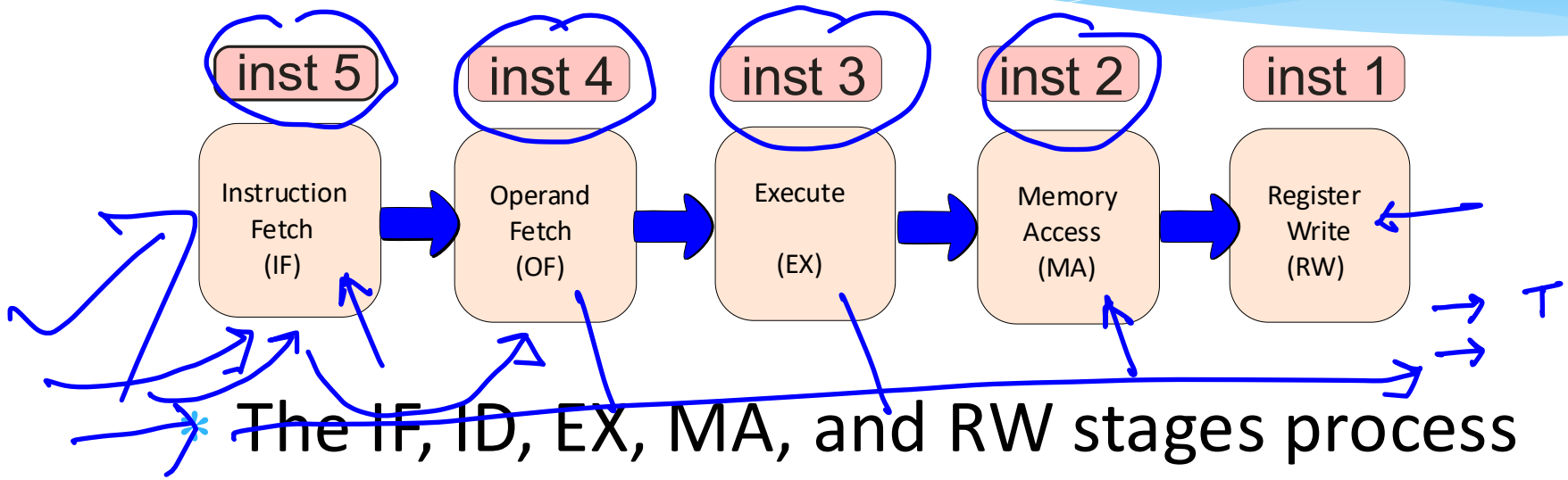
* Microprogrammed processors are much slower that hardwired processors
* Even hardwired processors
  * Have a lot of waste !!!
  * We have 5 stages.
  * What is the IF stage doing, when the MA stage is active ?
    * **ANSWER** : It is **idling**



The Notion of Pipelining

* Let us go back to the car assembly line
  * Is the engine shop idle, when the paint shop is painting a car ?
  * **NO** : It is building the engine of another car
  * When this engine goes to the body shop, it builds the engine of another car, and so on ....
  * **Insight** :
    * Multiple cars are built at the same time.
    * A car proceeds from one stage to the next

Pipelined Processors



* The IF, ID, EX, MA, and RW stages process 5 instructions simultaneously
* Each instruction proceeds from one stage to the next
* This is known as pipelining

Advantages of Pipelining

* We keep all parts of the data path, busy all the time
* Let us assume that all the 5 stages do the same amount of **work**
  * Without pipelining, every T seconds, an instruction completes its execution
  * With pipelining, every T/5 seconds, a new instruction completes its execution

Design of a Pipeline

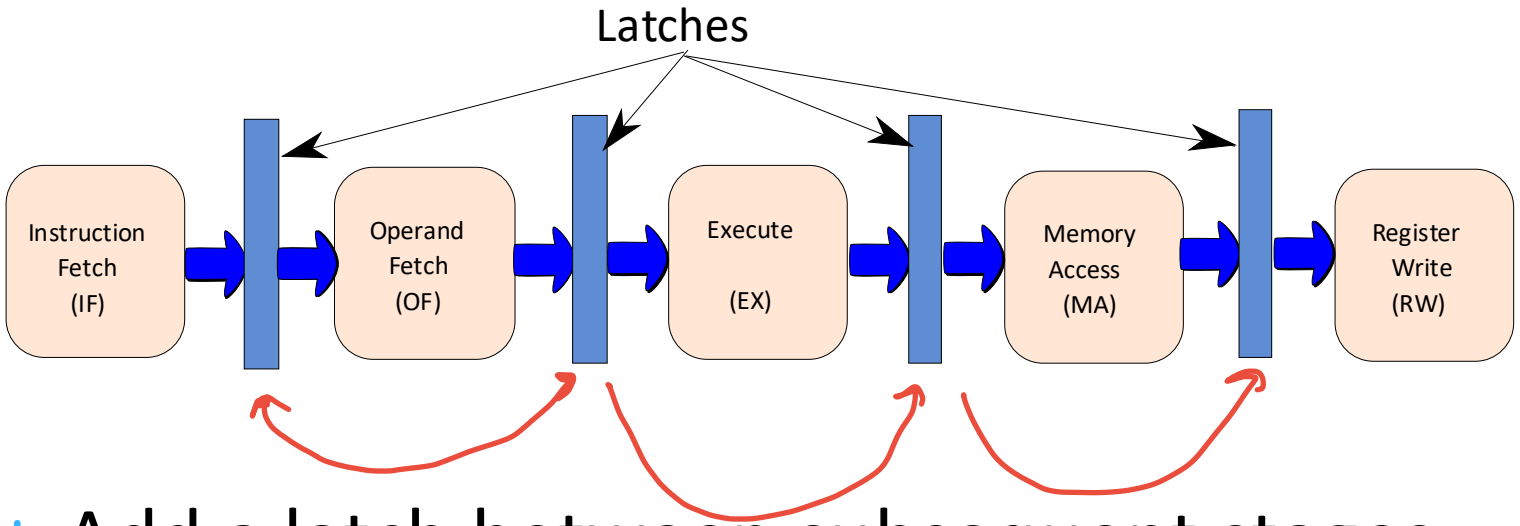
* Splitting the Data Path

  * We divide the data path into 5 parts : IF, OF, EX, MA, and RW

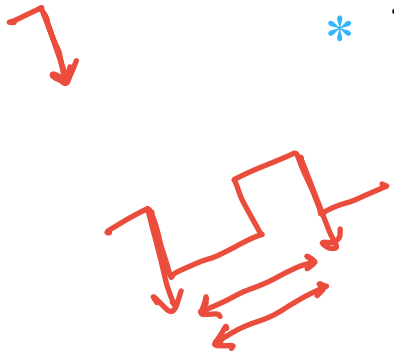
* Timing

  * We insert latches (pipeline registers) between consecutive stages
  * 4 Latches → IF-OF, OF-EX, EX-MA, and MA-RW
  * At the negative edge of a clock, an instruction moves from one stage to the next

Pipelined Data Path with Latches



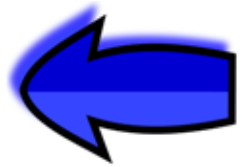
* Add a latch between subsequent stages.
  * Triggered by a negative clock edge



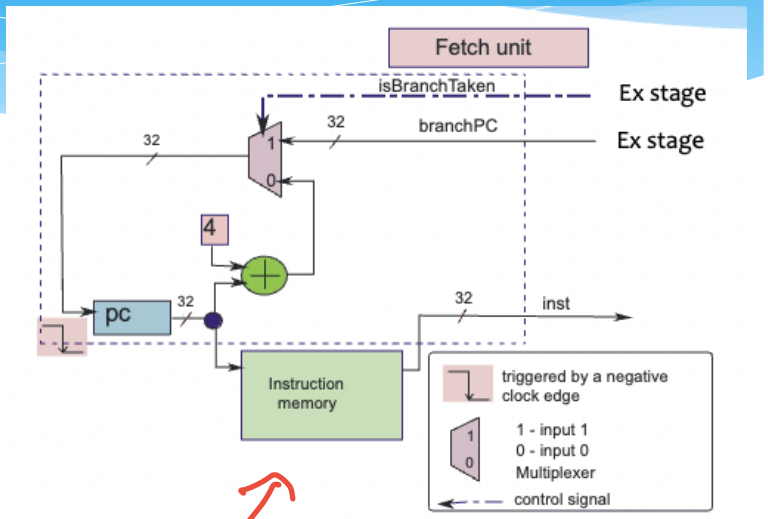
The Instruction Packet

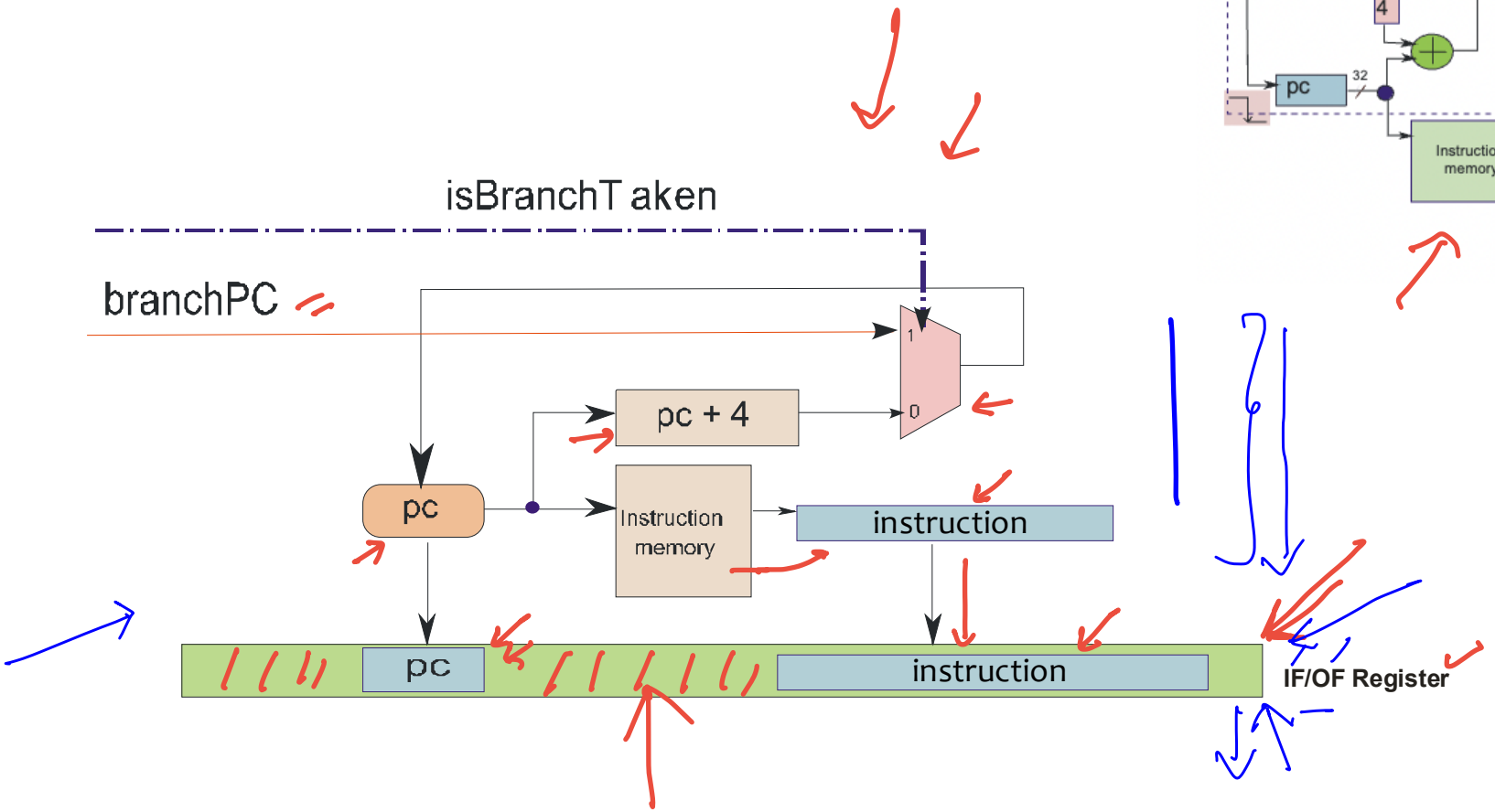
- What travels between stages ?
  - **ANSWER** : the instruction packet
- Instruction Packet
  - Instruction contents
  - Program counter
  - All intermediate results
  - Control signals
- Every instruction moves with its entire state, no interference between instructions

Outline

* Overview of Pipelining
* A Pipelined Data Path 
* Pipeline Hazards
* Pipeline with Interlocks
* Forwarding
* Performance Metrics
* Interrupts/ Exceptions

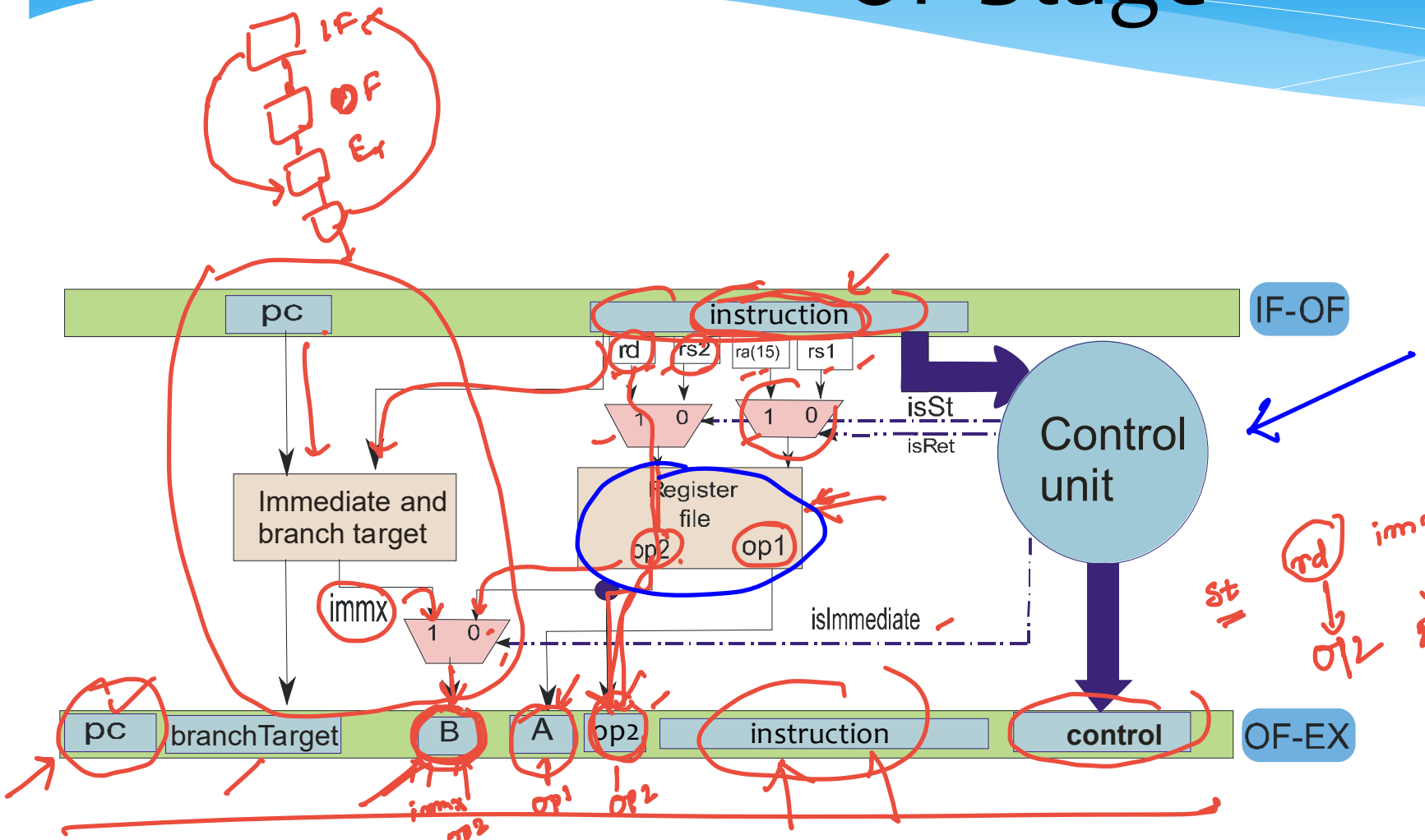
IF Stage

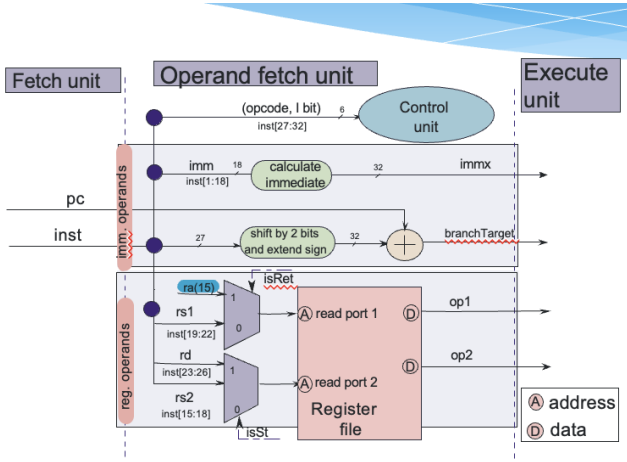




* Instruction contents saved in the instruction field

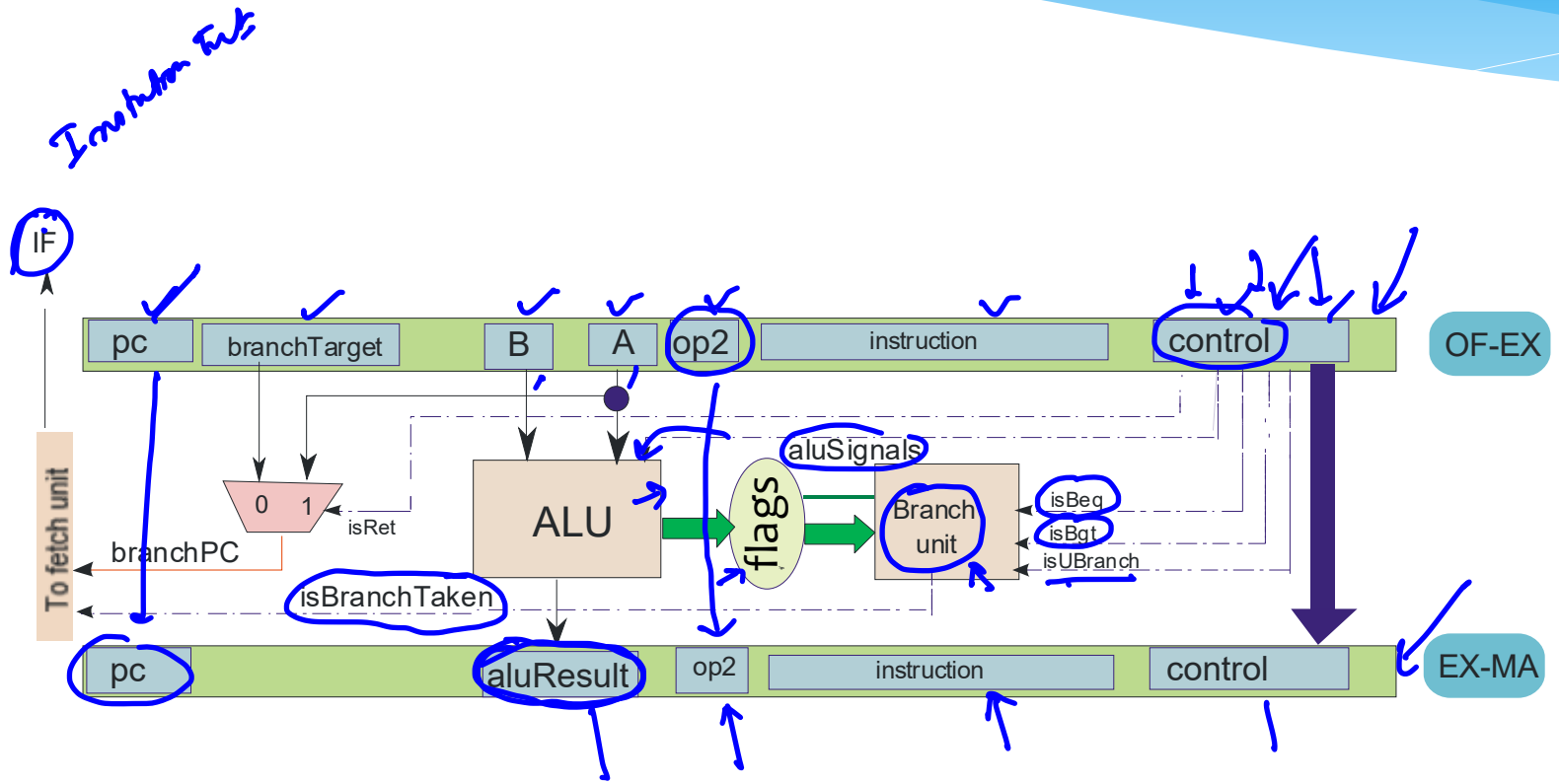
OF Stage

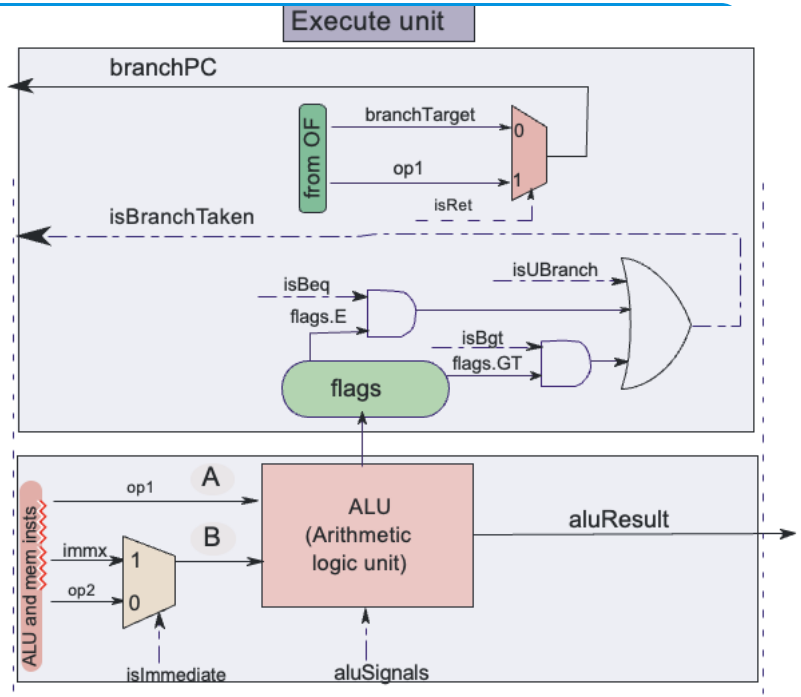




* A, B → ALU Operands, op2 (store operand), control (set of all control signals)

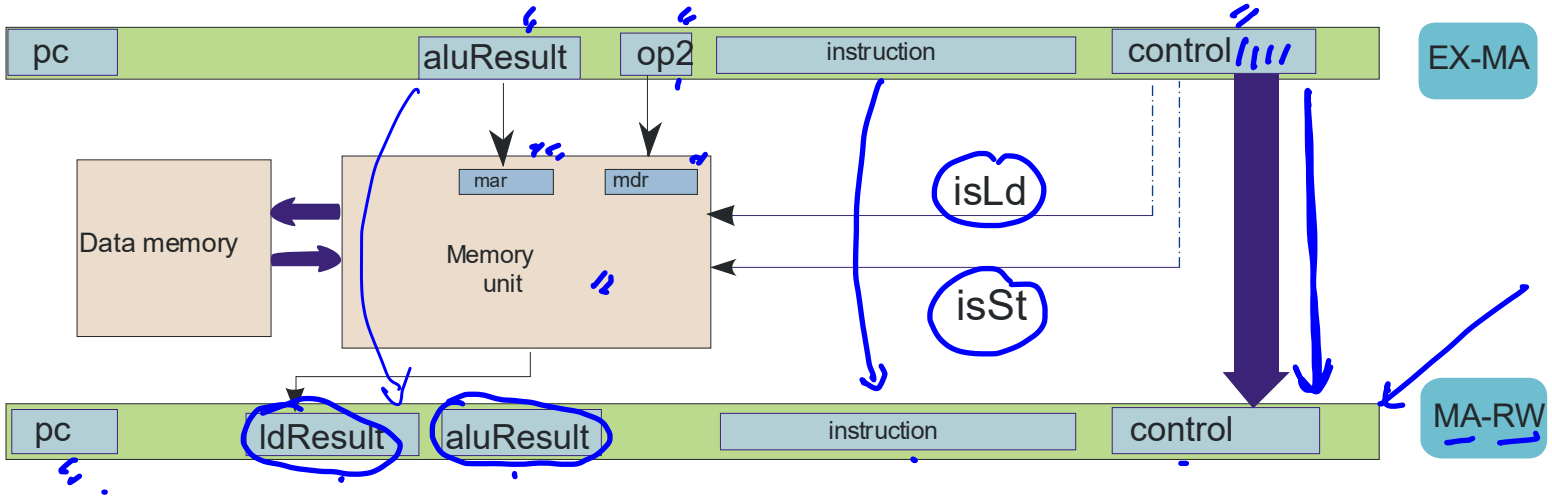
EX Stage

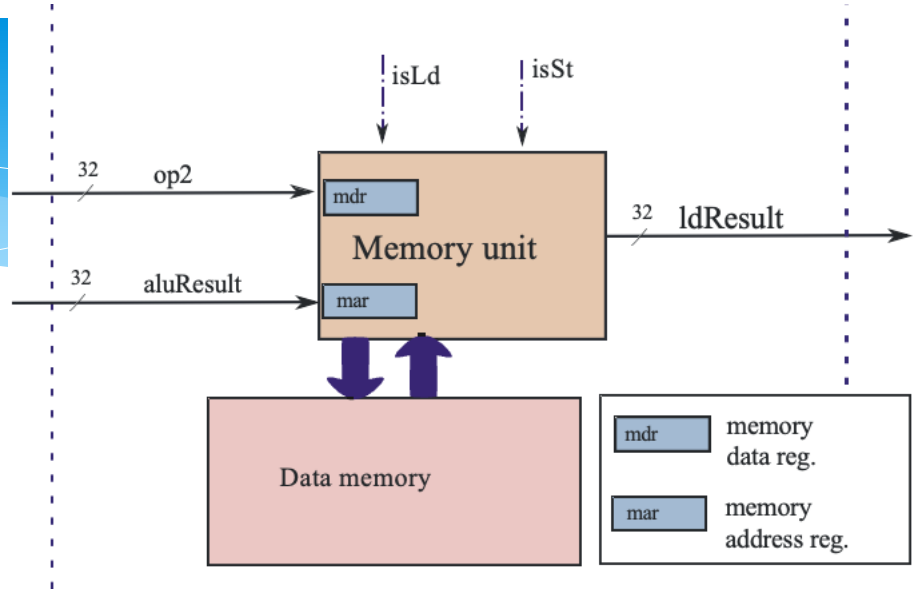




* aluResult → result of the ALU Operation
* op2, control, pc, instruction (passed from OF-EX)

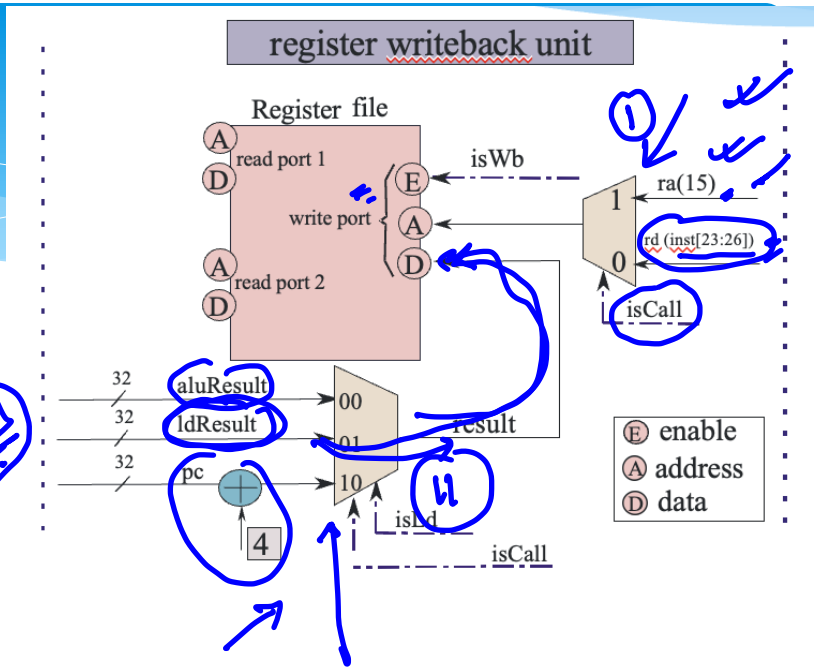
MA Stage

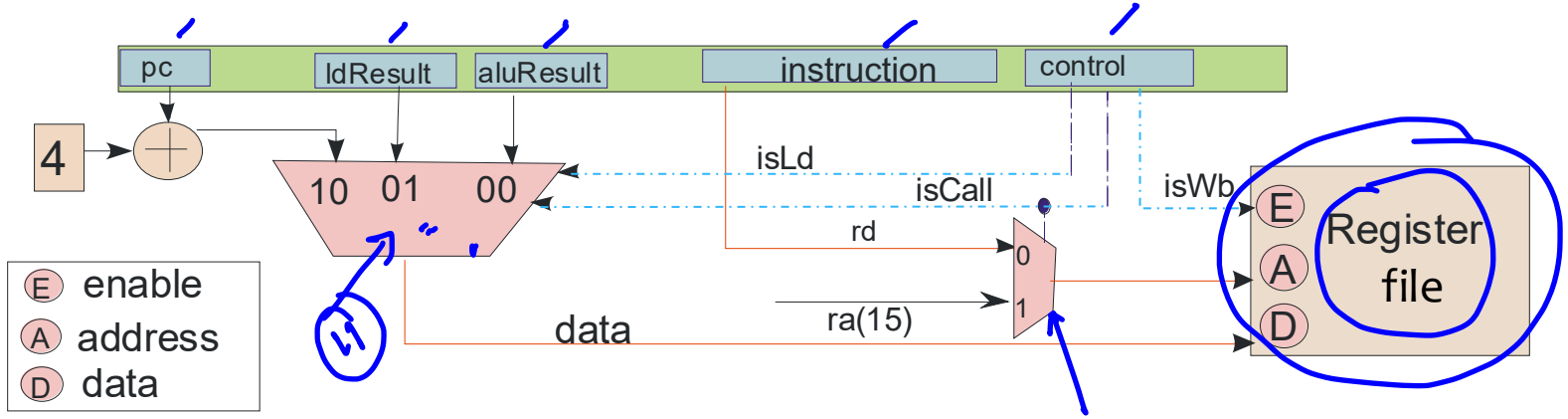


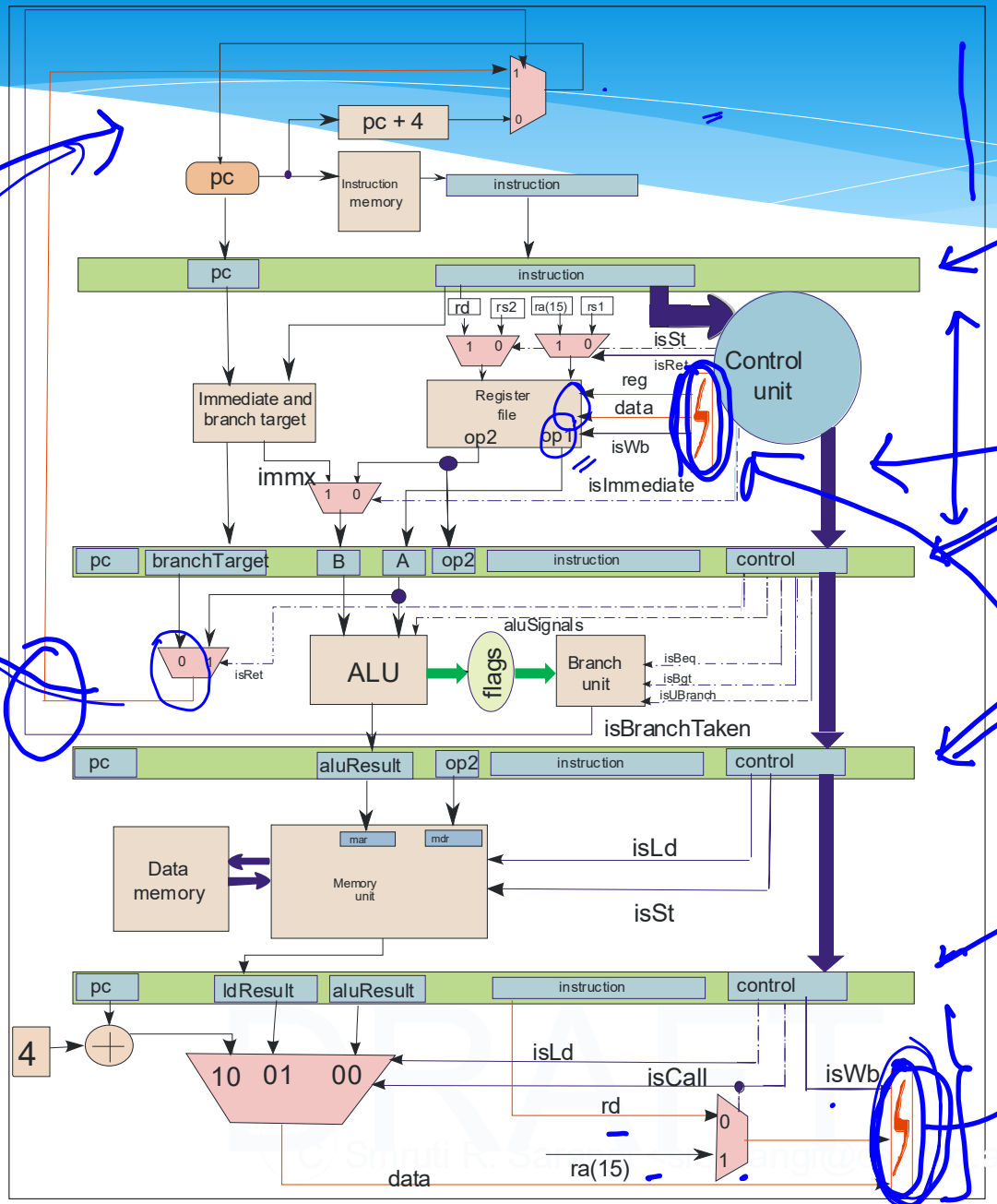


* ldResult → result of the load operation
* aluResult, control, pc, instruction (passed from EX-MA)

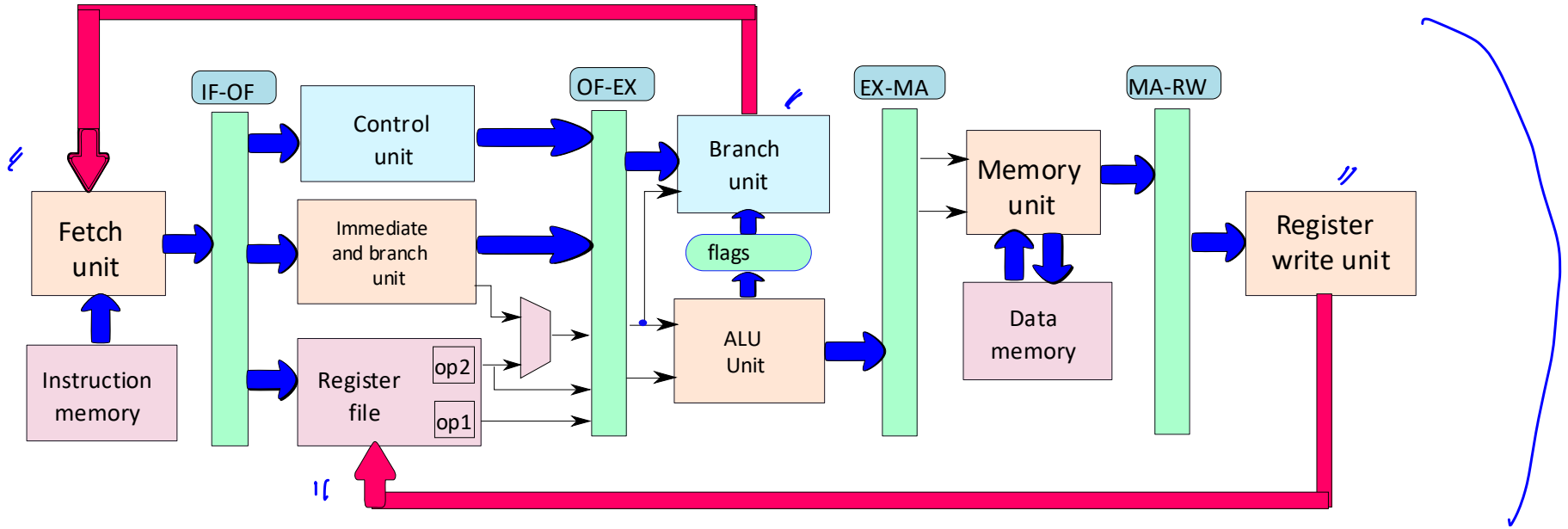
RW Stage


register writeback unit
Register file
read port 1
read port 2
write port
isWb
ra(15)
rd (inst[23:26])
isCall
32
aluResult
ldResult
pc
00
01
10
result
isLd
isCall
4
enable
address
data


pc
ldResult
aluResult
instruction
control
4
10 01 00
isLd
isCall
isWb
rd
ra(15)
data
0
1
E
A
D
Register file
enable
address
data

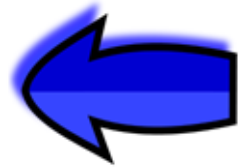

pc + 4
pc
Instruction memory
instruction
rd
rs2
ra(15)
rs1
isSt
isRet
Control unit
Immediate and branch target
Register file
reg
data
op2
op1
isWb
immx
isImmediate
branchTarget
B
A
control
aluSignals
ALU
flags
Branch unit
isBeq
isBgt
isUBranch
isBranchTaken
aluResult
Data memory
mar
mdr
Memory unit
isLd
ldResult
4
10 01 00
isCall
ra(15)
OF
RW

Abridged Diagram



Outline

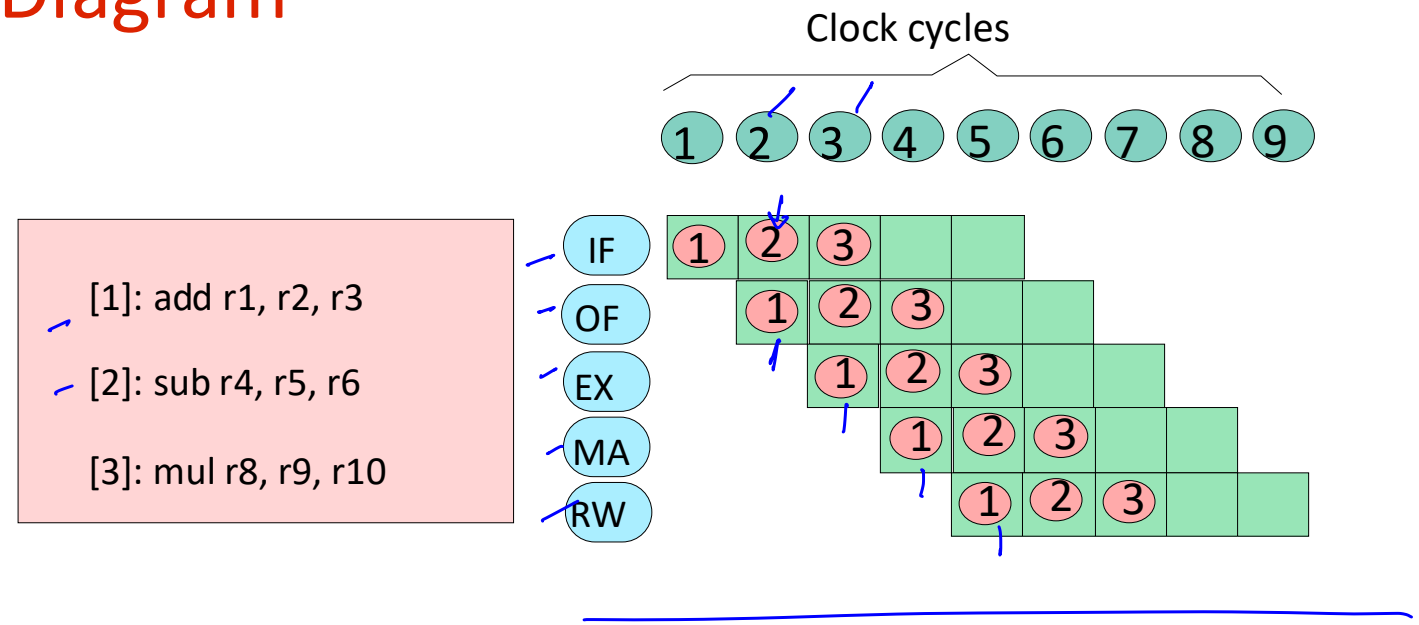
* Overview of Pipelining
* A Pipelined Data Path
* Pipeline Hazards
* Pipeline with Interlocks
* Forwarding
* Performance Metrics
* Interrupts/ Exceptions



Pipeline Hazards

* Now, let us consider correctness
* Let us introduce a new tool → Pipeline Diagram

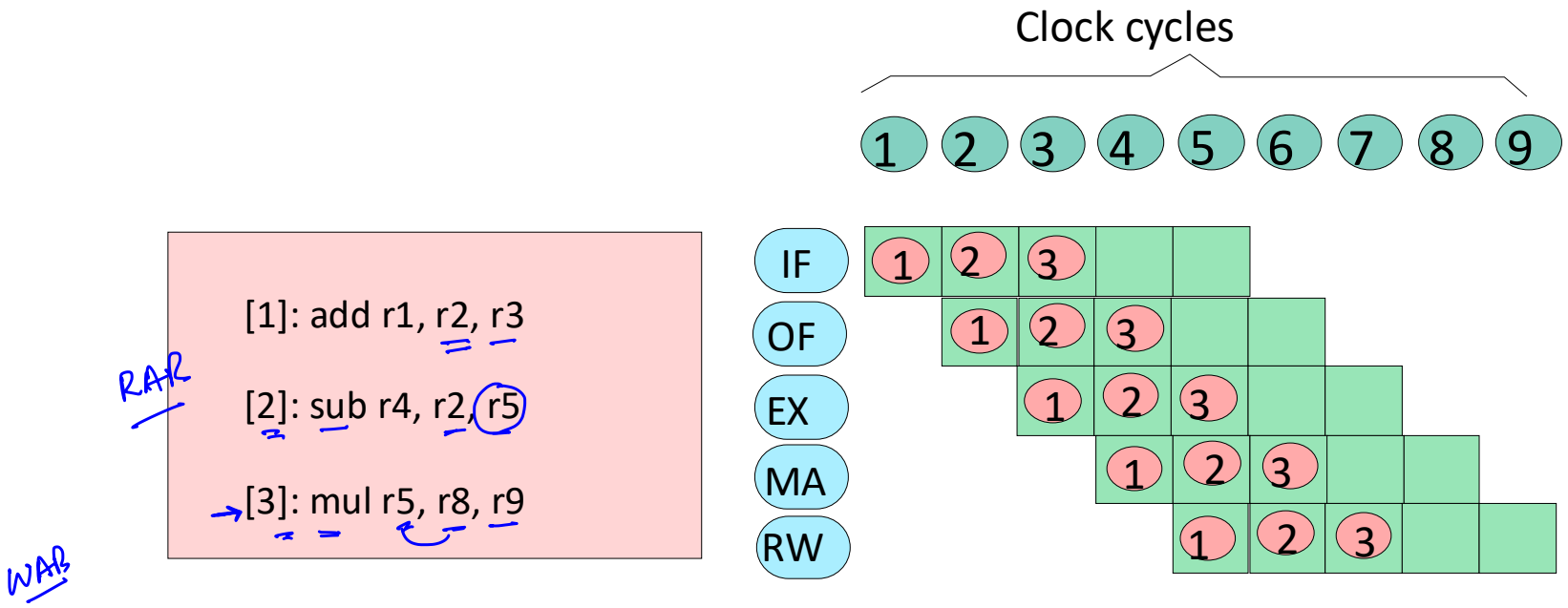
RAW



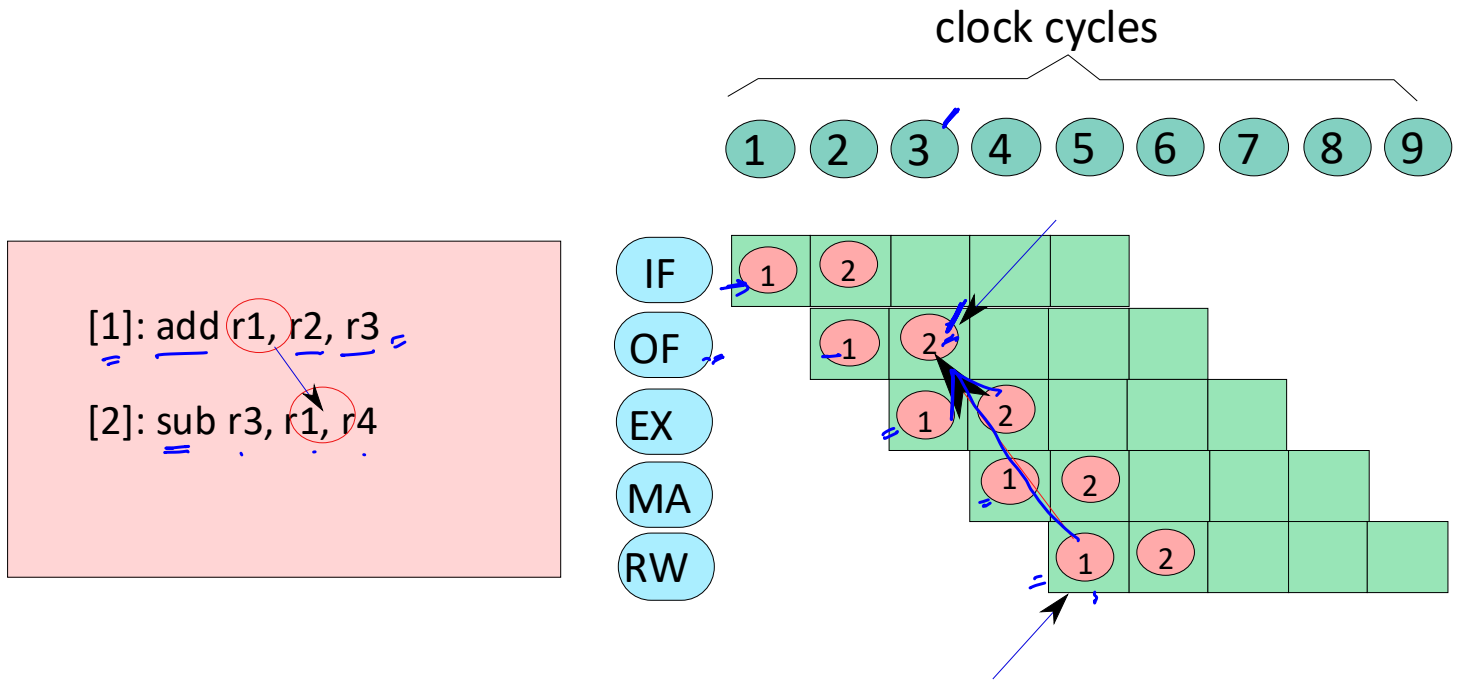
Rules for Constructing a Pipeline Diagram

* It has 5 rows
  * One per each stage
  * The rows are named : IF, OF, EX, MA, and RW
* Each column represents a clock cycle
* Each cell represents the execution of an instruction in a stage
  * It is annotated with the name(label) of the instruction
* Instructions proceed from one stage to the next across clock cycles

Example


Clock cycles
1 2 3 4 5 6 7 8 9
[1]: add r1, r2, r3
[2]: sub r4, r2, r5
[3]: mul r5, r8, r9
RAR
WAR
IF
OF
EX
MA
RW

Data Hazards



RAW ⑤

* Instruction 2 will read incorrect values !!!

Data Hazard

Definition: A hazard is defined as the possibility of erroneous execution of an instruction in a pipeline. A data hazard represents the possibility of erroneous execution because of the unavailability of data, or the availability of incorrect data.

* This situation represents a data hazard
* In specific,
  * it is a RAW (read after write) hazard
* The earliest we can dispatch instruction 2, is cycle 5

Other Types of Data Hazards

* Our pipeline is in-order

Definition: In an **in-order** pipeline (such as ours), a preceding instruction is always ahead of a succeeding instruction in the pipeline. Modern processors however use out-of-order pipelines that break this rule. It is possible for later instructions to execute before earlier instructions.

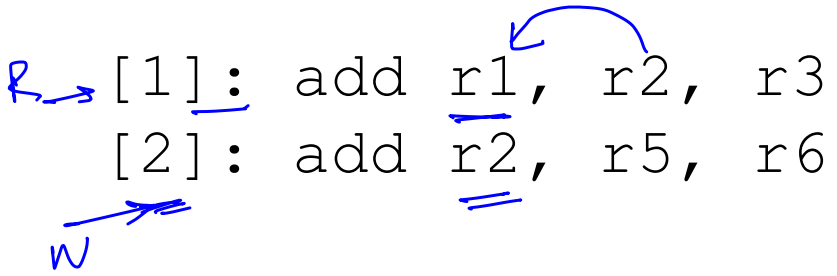
* We will only have RAW hazards in our pipeline.
* Out-of-order pipelines can have WAR and WAW hazards

WAW Hazards

```
[1]: add r1, r2, r3
[2]: sub r1, r4, r3
```

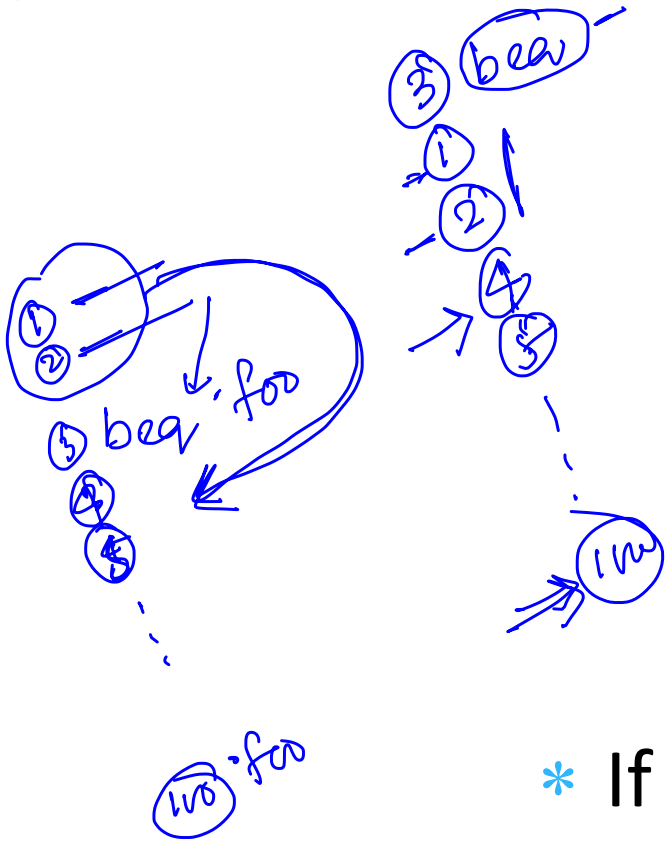
* Instruction [2] cannot write the value of r1, before instruction [1] writes to it, will lead to a WAW hazard

WAR Hazards

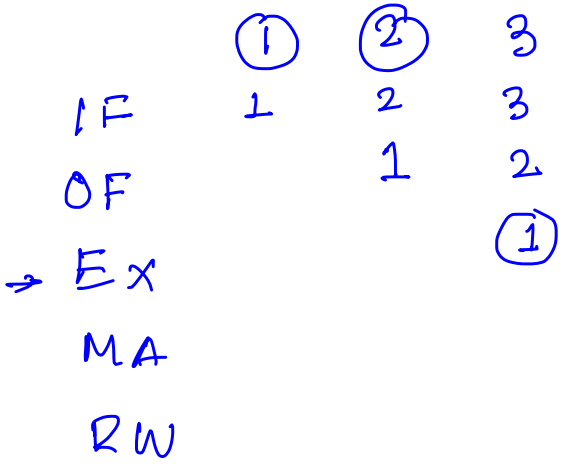


* Instruction [2] cannot write the value of r2, before instruction [1] reads it → will lead to a WAR hazard

Control Hazards

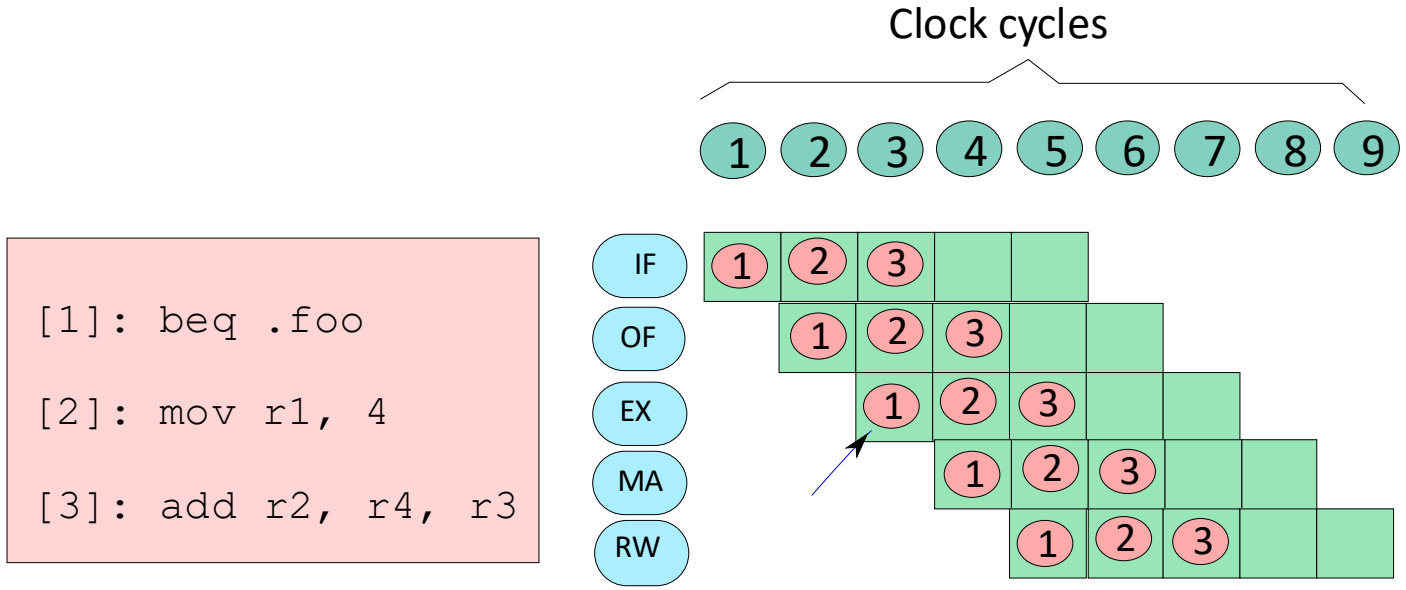


```
[1]: beq .foo
[2]: mov r1, 4
[3]: add r2, r4, r3
...
...
.foo:
[100]: add r4, r1, r2
```



* If the branch is taken, instructions [2] and [3], might get fetched, incorrectly

Control Hazard – Pipeline Diagram



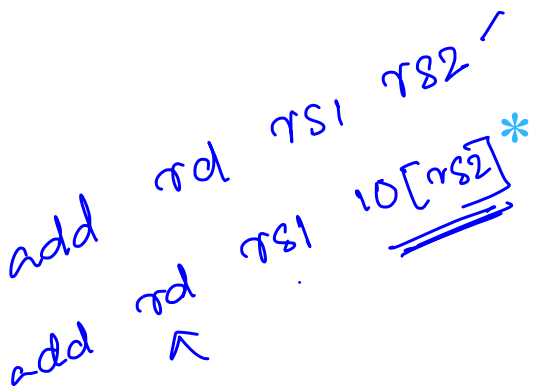
* The two instructions fetched immediately after a branch instruction might have been fetched incorrectly.

Control Hazards

* The two instructions fetched immediately after a branch instruction might have been fetched incorrectly.
* These instructions are said to be on the wrong path
* A control hazard represents the possibility of erroneous execution in a pipeline because instructions in the wrong path of a branch can possibly get executed and save their results in memory, or in the register file

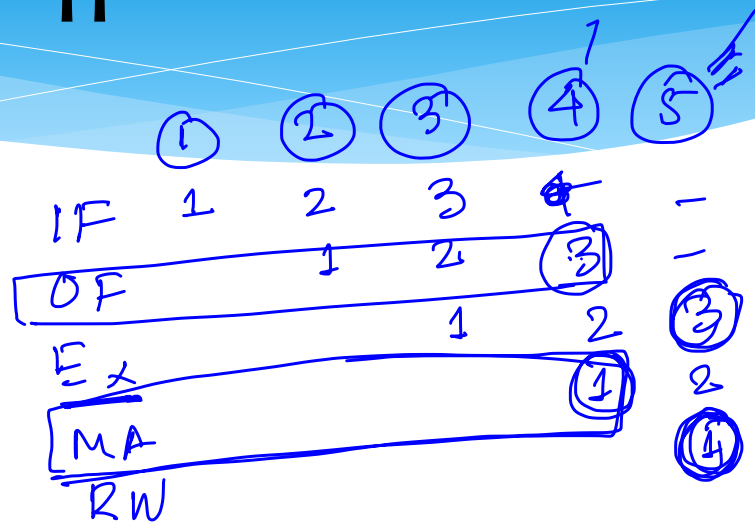
Structural Hazards

* A structural hazard may occur when two instructions have a conflict on the same set of resources in a cycle
* Example :
  * Assume that we have an add instruction that can read one operand from memory
  * add r1, r2, 10[r3]



Structural Hazards - II

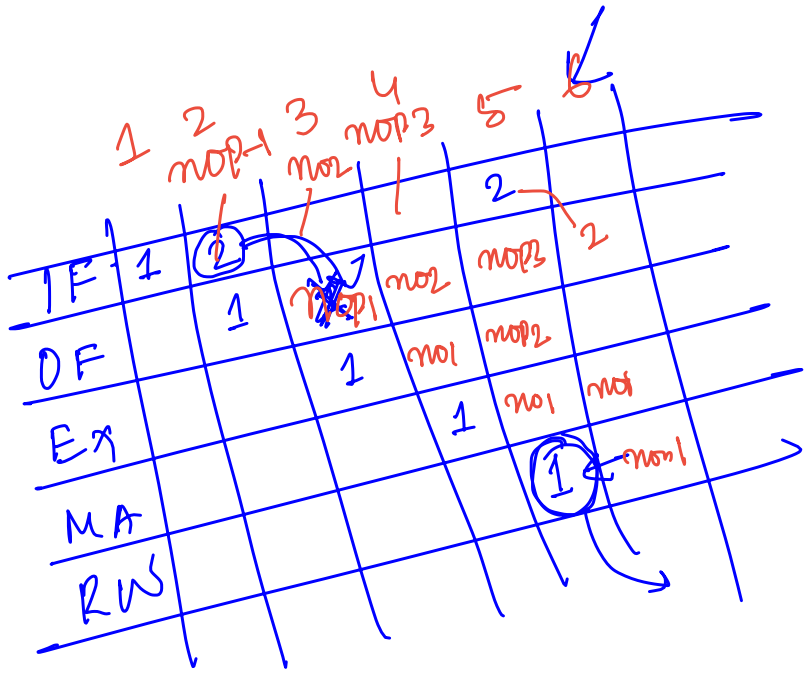
```
[1]: st  r4, 20[r5]
[2]: sub r8, r9, r10
[3]: add r1, r2, 10[r3]
```



* This code will have a structural hazard
  * [3] tries to read 10[r3] (MA unit) in cycle 4
  * [1] tries to write to 20[r5] (MA unit) in cycle 4
* Does not happen in our pipeline

Solutions in Software

* Data hazards
  * Insert nop instructions, reorder code

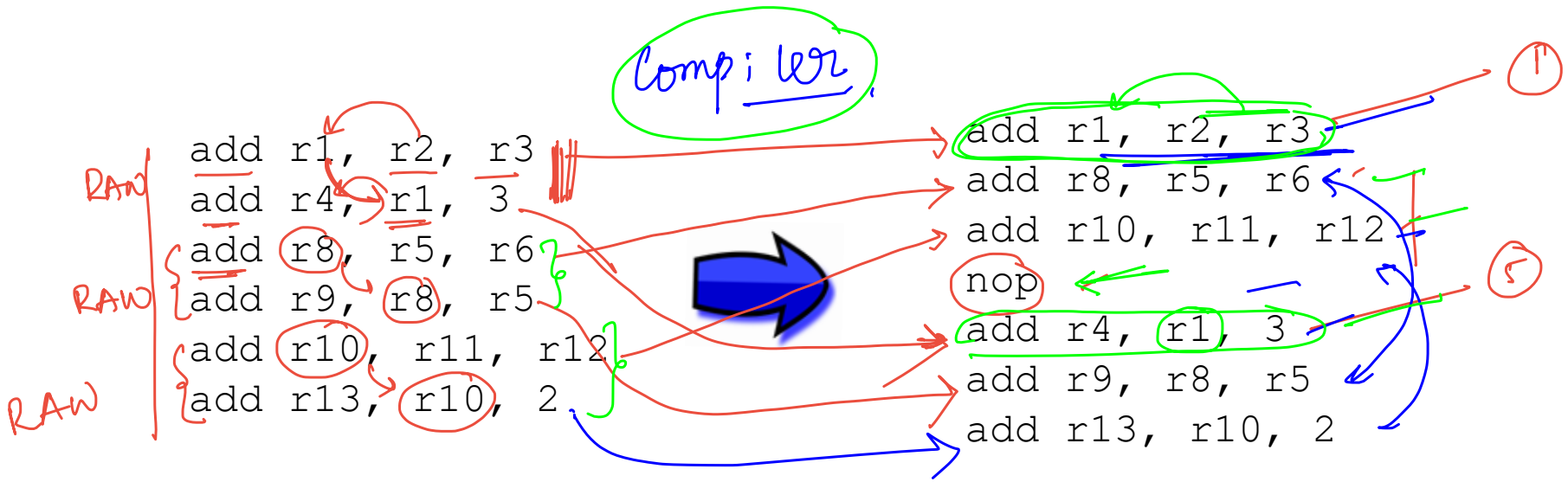



```
[1]: add r1, r2, r3
[2]: sub r3, r1, r4
```

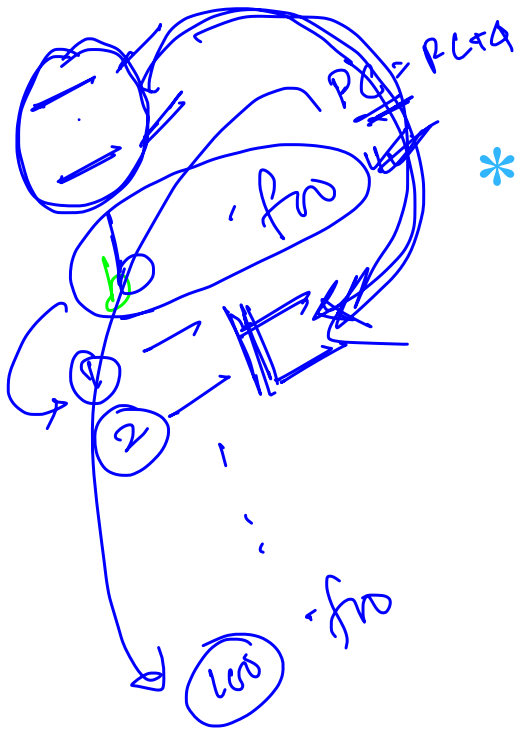
RAW

```
[1]: add r1, r2, r3
[2]: nop
[3]: nop
[4]: nop
[5]: sub r3, r1, r4
```

Code Reordering



Control Hazards



* Trivial Solution : Add two nop instructions after every branch
* Better solution :
  * Assume that the two instructions fetched after a branch are valid instructions
  * These instructions are said to be in the delay slots
  * Such a branch is known as a delayed branch

Example with 2 Delay Slots

```
add r1, r2, r3
add r4, r5, r6
b .foo
add r8, r9, r10
```

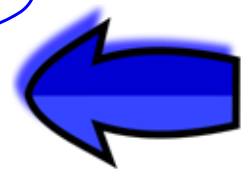
```
b .foo
add r1, r2, r3
add r4, r5, r6
add r8, r9, r10
```

* The compiler transfers instructions before the branchto the delay slots.

* If it cannot find 2 valid instructions, it inserts nops.

Outline

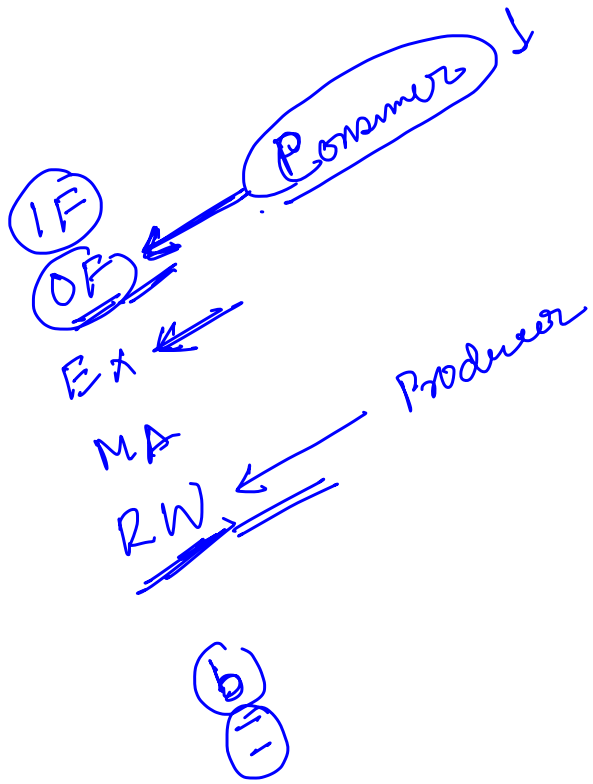
* Overview of Pipelining
* A Pipelined Data Path
* Pipeline Hazards
* Pipeline with Interlocks
* Forwarding
* Performance Metrics
* Interrupts/ Exceptions



Why interlocks ?

* We cannot always trust the compiler to do a good job, or even introduce nop instructions correctly.
* Compilers now need to be tailored to specific hardware.
* We should ideally not expose the details of the pipeline to the compiler (might be confidential also)
* Hardware mechanism to enforce correctness → **interlock**

Two kinds of Interlocks



* Data-Lock
  * Do not allow a consumer instruction to move beyond the OF stage till it has read the correct values. Implication : **Stall** the IF and OF stages.
* Branch-Lock
  * We never execute instructions in the wrong path.
* The hardware needs to ensure both these conditions.

Comparison between Software and Hardware

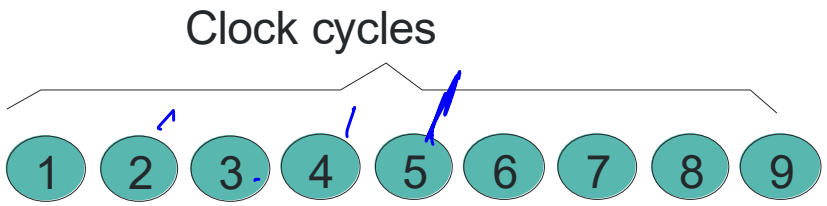
Attribute	Software	Hardware(withinterlocks)
Portability	Limited to a specific processor	Programs can be run on any processor irrespective of the nature of the pipeline
Branches	Possible to have no performance penalty, by using delay slots	Need to stall the pipeline for 2 cycles in our design
RAW hazards	Possible to eliminate them through code scheduling	Need to stall the pipeline
Performance	Highly dependent on the nature of the program	The basic version of a pipeline with interlocks is expected to be slower than the version that relies on software

Conceptual Look at Pipeline with Interlocks

```
[1]: add r1, r2, r3
[2]: sub r4, r1, r2
```

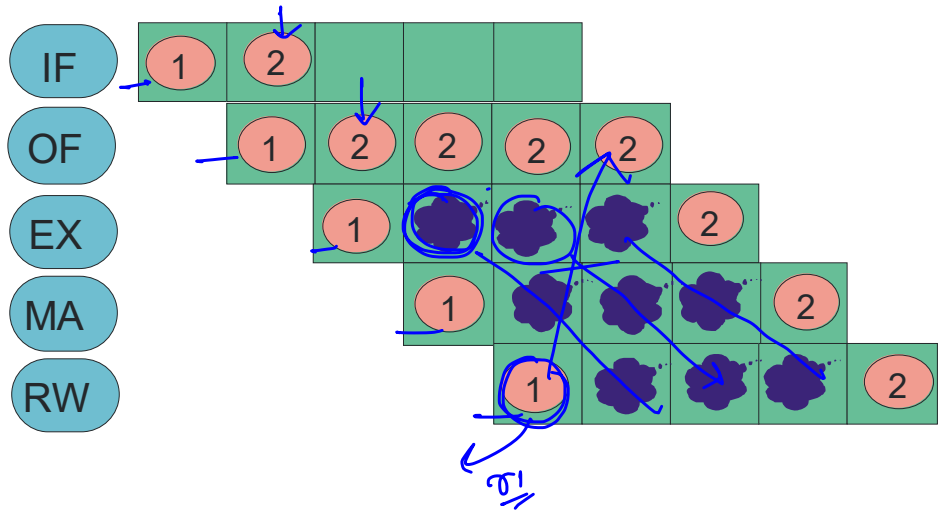
* We have a RAW hazard
* We need to stall, instruction [2] at the OF stage for 3 cycles.
* We need to keep sending nop instructions to the EX stage during these 3 cycles

Example

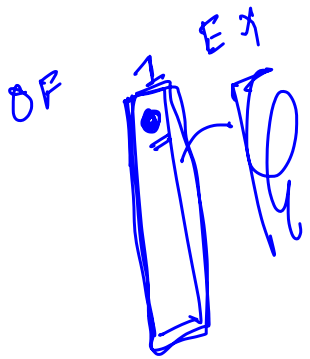


[1]: add r1, r2, r3

[2]: sub r4, r1, r2

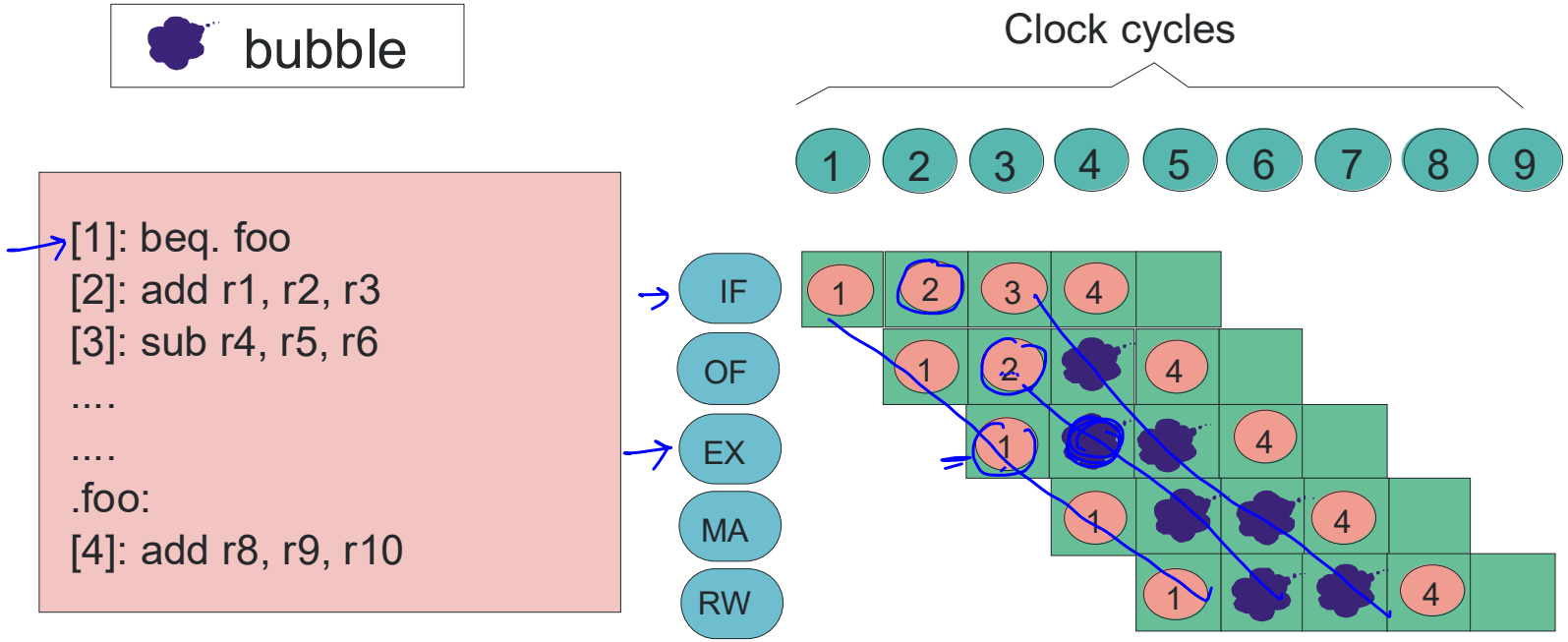


A Pipeline Bubble



* A pipeline bubble is inserted into a stage, when the previous stage needs to be stalled
* It is a nop instruction
* To insert a bubble
  * Create a nop instruction packet
  * OR, Mark a designated bubble bit to 1

Bubbles in the Case of a Branch Instruction



Control Hazards and Bubbles

* We know that an instruction is a branch in the OF stage
* When it reaches the EX stage and the branch is taken, let us convert the instructions in the IF, and OF stages to bubbles
* Ensures the branch-lock condition

Ensuring the Data-Lock Condition

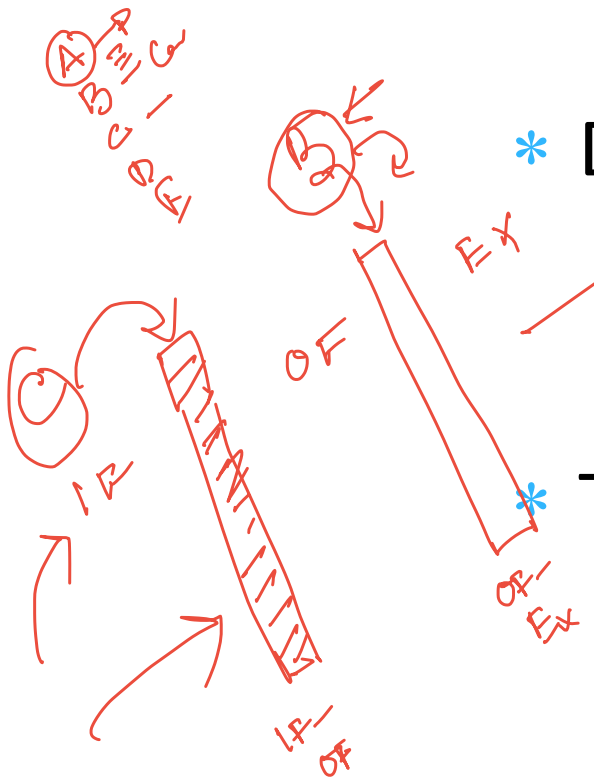
* When an instruction reaches the OF stage, check if it has a conflict with any of the instructions in the EX, MA, and RW stages
* If there is **no** conflict, nothing needs to be done
* Otherwise, stall the pipeline (IF and OF stages only)

Algorithm 5: Algorithm to detect conflicts between instructions

Data: instructions, [A], and [B]
Result: conflict exists (**true**), no conflict (**false**)
if *[A].opcode ∈ (nop,b,beq,bgt,call)* **then**
    /* Does not read from any register */
    return false
end
if *[B].opcode ∈ (nop, cmp, st, b, beq, bgt, ret)* **then**
    /* Does not write to any register */
    return false
end
    /* Set the sources */
src1 ← [A].rs1
src2 ← [A].rs2
if *[A].opcode = st* **then**
    src2 ← [A].rd
end
if *[A].opcode = ret* **then**
    src1 ← *ra*
end
hasSrc1 ← ***true***
if ([A] ∈ (not, mov)) hasSrc1 ← ***false***

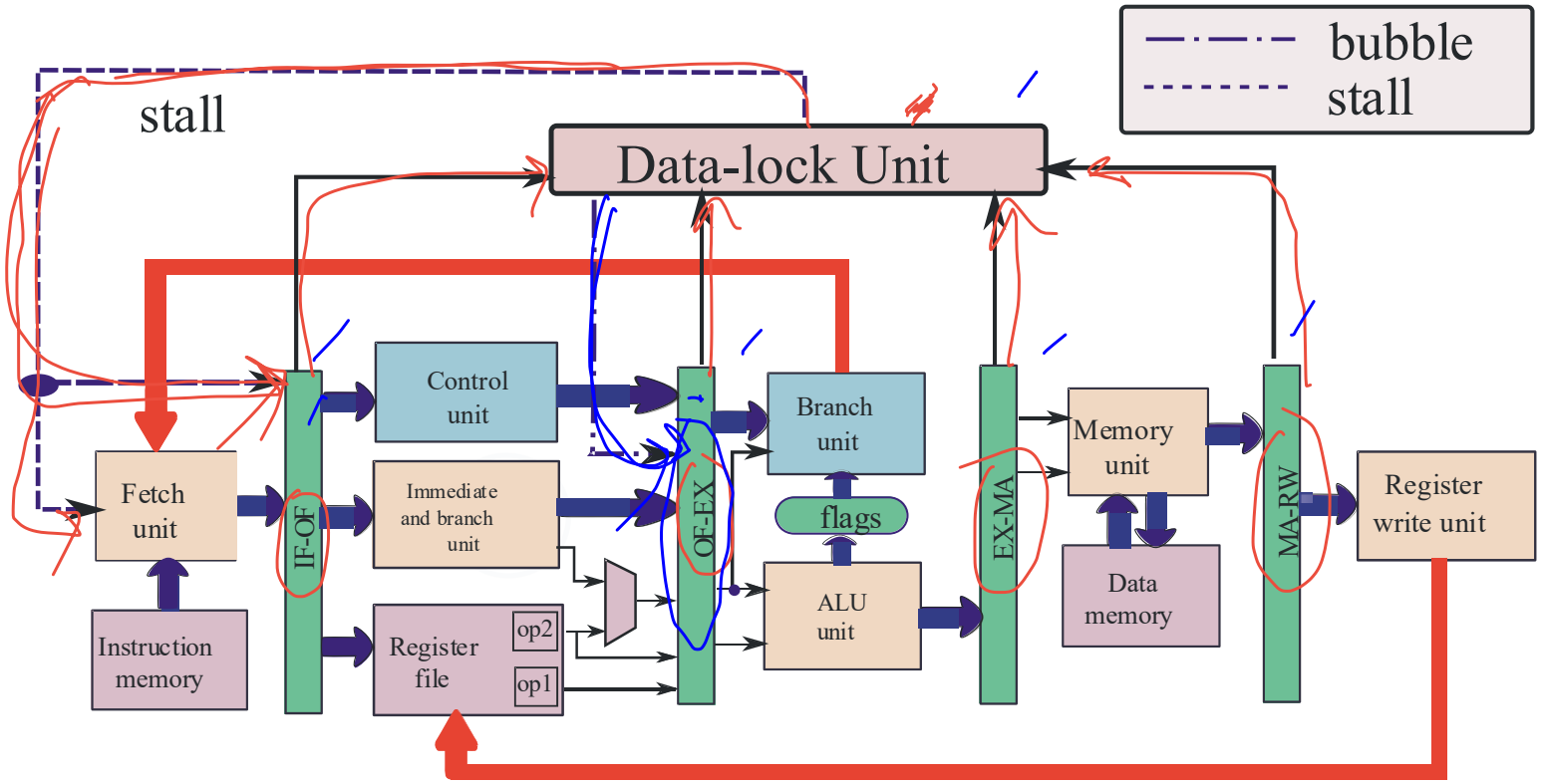
```
dest ← [B].rd
if [B].opcode = call then
        dest ← ra
end
/* Check the second operand to see if it is a register                    */
hasSrc2 ← true
if [A].opcode  ≠ ( st) then
        if [A].I = 1 then
                hasSrc2 ← false
        end
end
/* Detect conflicts  */                                                   */
if  (hasSrc1 = true) and (src1 = dest) then
        return true
end
else if (hasSrc2 = true) and (src2 = dest) then
        return true
end
return false
```

How to Stall a Pipeline ?



* Disable the write functionality of :
  * The IF-OF register
  * and the Program Counter (PC)
* To insert a bubble
  * Write a bubble (nop instruction) into the OF-EX register

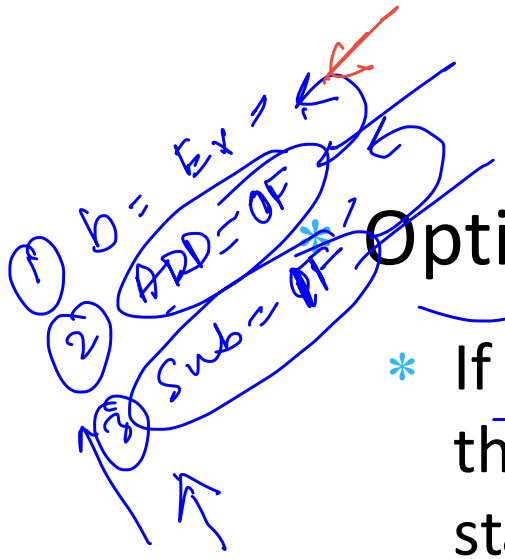
Data Path with Interlocks (Data-Lock)



Ensuring the Branch-Lock Condition

* Option 1 :
  * Use delay slots (interlocks not required)
* Option 2 :
  * Convert the instructions in the IF, and OF stages, to bubbles once a branch instruction reaches the EX stage.
  * Start fetching from the next PC (not taken) or the branch target (taken)

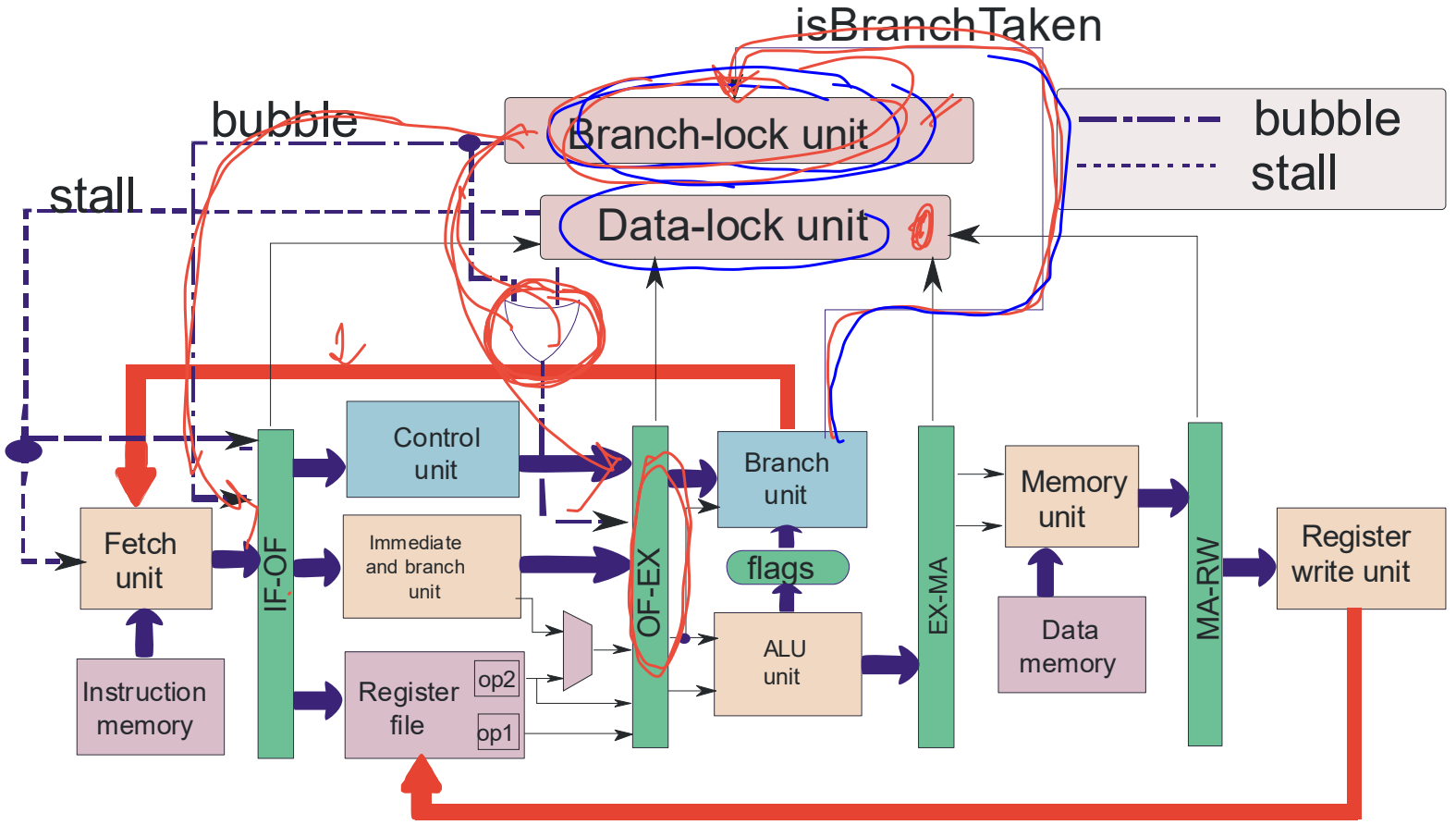
Ensuring the Branch-Lock Condition - II



Option 3

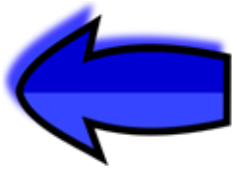
- If the branch instruction in the EX stage is **taken**, then invalidate the instructions in the IF and OF stages. Start fetching from the branch target.
- Otherwise, do not take **any special action**
- **This method is also called predict not-taken (we shall use this method because it is more efficient that option 2)**

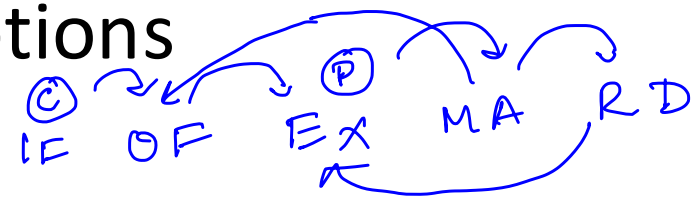
Data Path with Interlocks



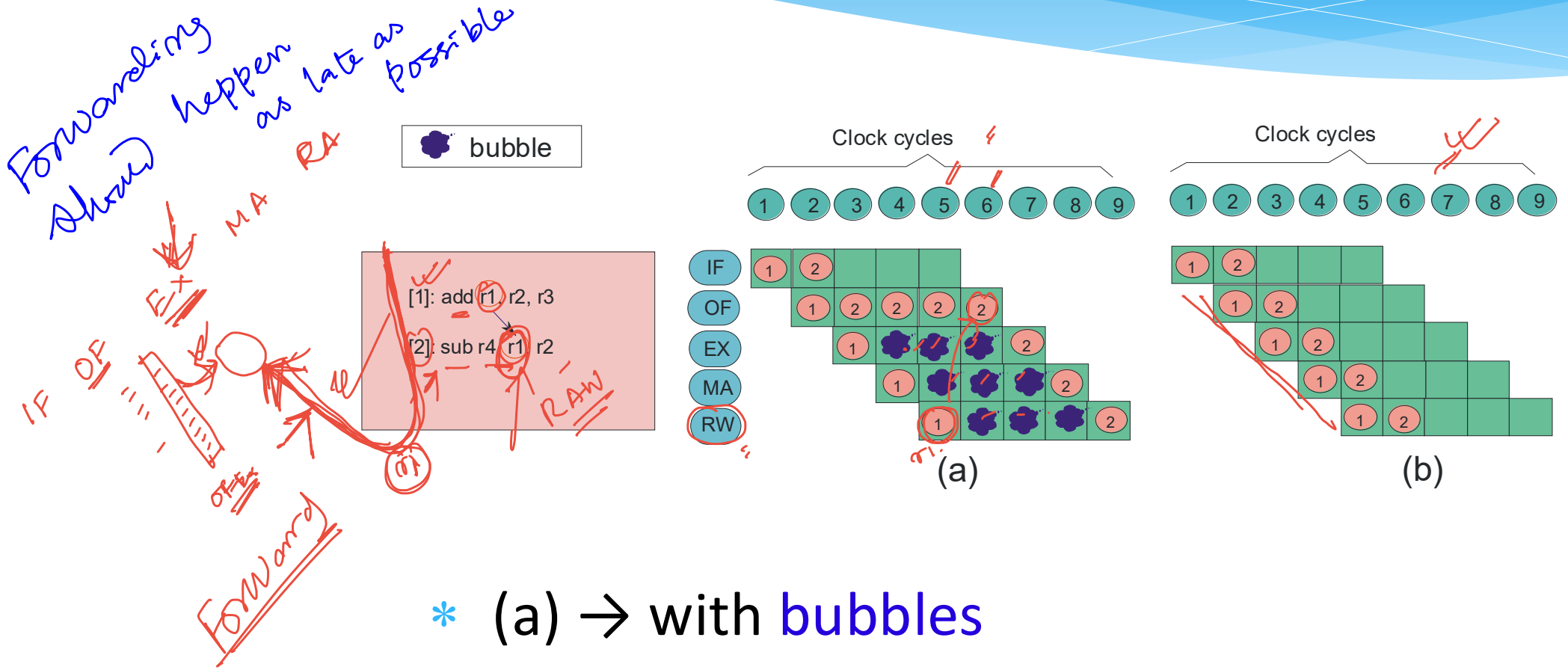
Outline

* Overview of Pipelining
* A Pipelined Data Path
* Pipeline Hazards
* Pipeline with Interlocks
* Forwarding
* Performance Metrics
* Interrupts/ Exceptions





Relook at the Pipeline Diagram

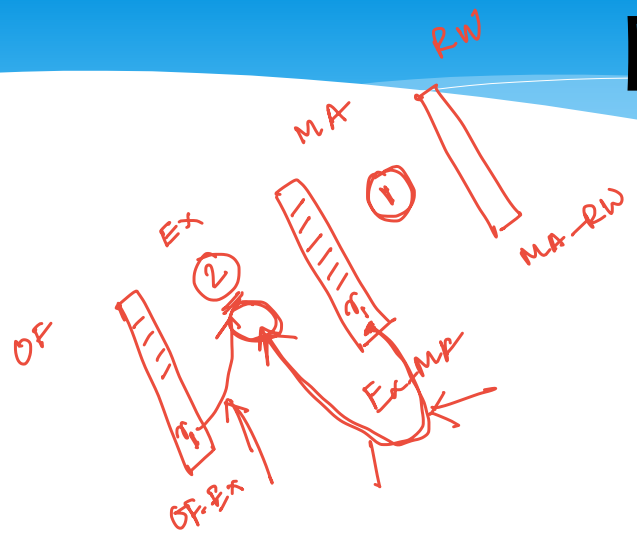


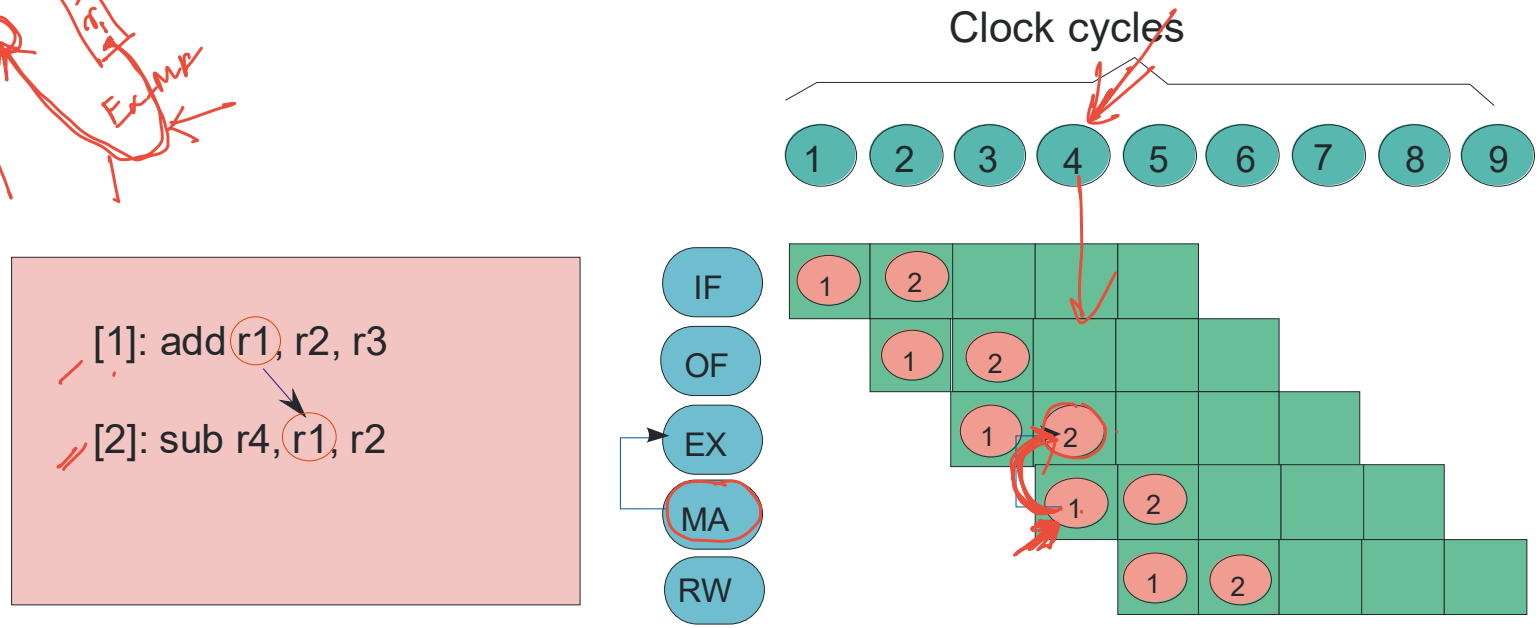
* (a) → with bubbles
* (b) → no bubbles (may lead to wrong results)

Crucial Insight (Figure (b))

* When does instruction 2 need the value of r1 ?
  * ANSWER : Cycle 3, OF Stage (wrong!!!)
  * **CORRECT ANSWER** : Cycle 4, EX Stage
* When does instruction 1 produce the value of r1 ?
  * ANSWER : Cycle 5, RW Stage (wrong!!!)
  * **CORRECT ANSWER** : End of Cycle 3, EX Stage

Forwarding

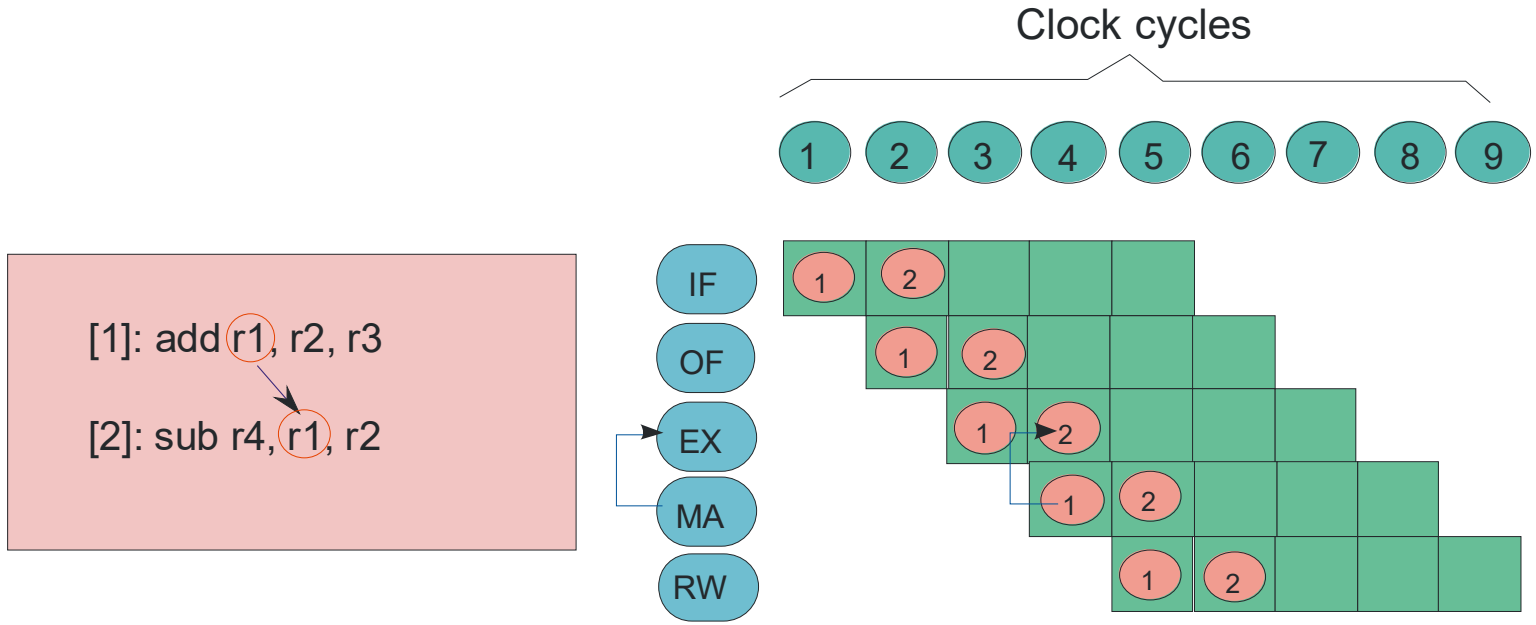




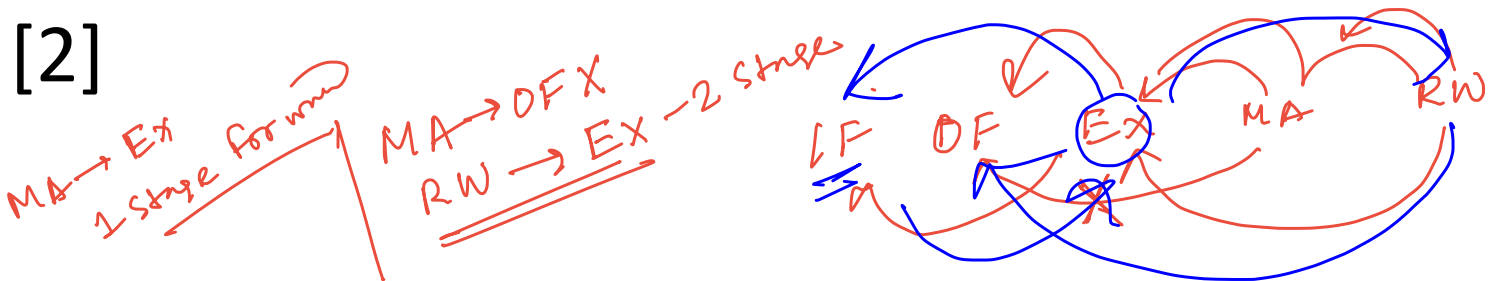
* If the correct value is already there in another stage, we can **forward** it.

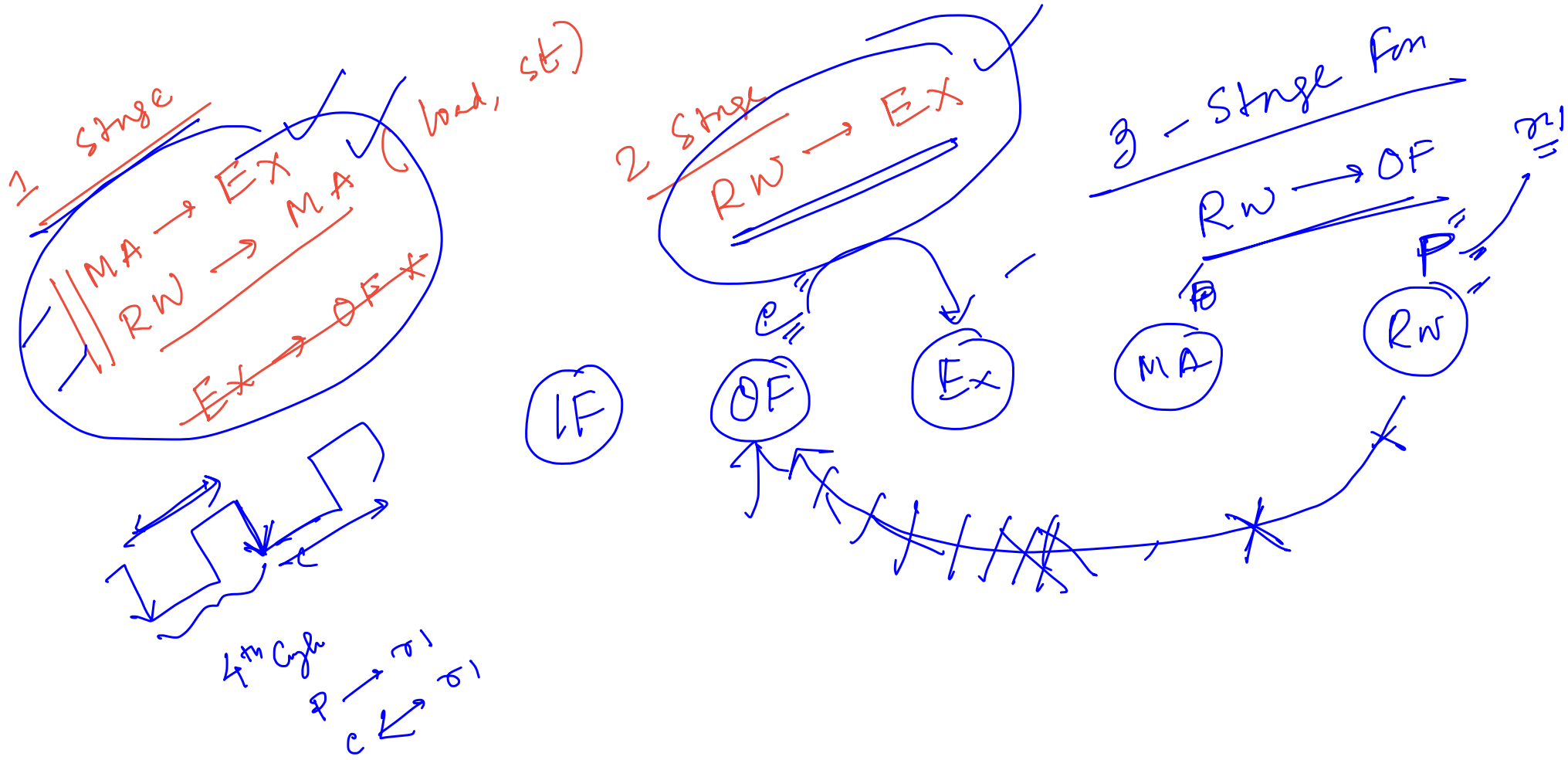
MA → EX

Forwarding from MA to EX



* Fowarding in cycle 4 from instruction [1] to [2]



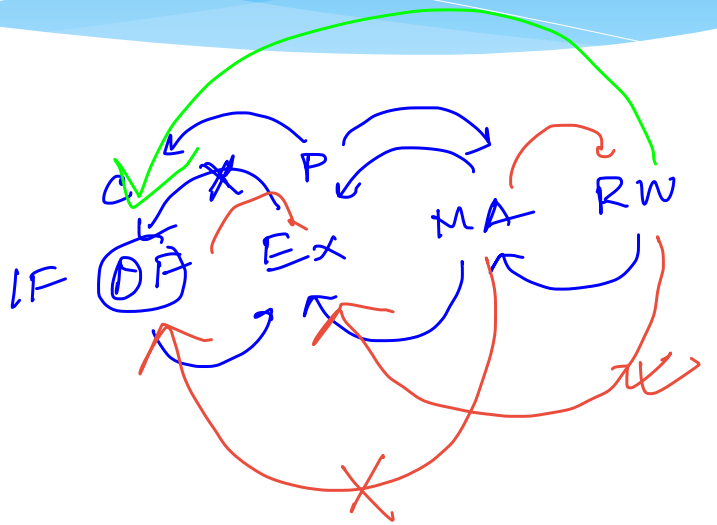

2 Stage
MA → EX
RW → MA
EX → OF
(load, st)
2 Stage
RW → EX
3 - Stage fn
RW → OF
IF
OF
EX
MA
RW
4th Cycle
P → σ1
C → σ1

Different Forwarding Paths

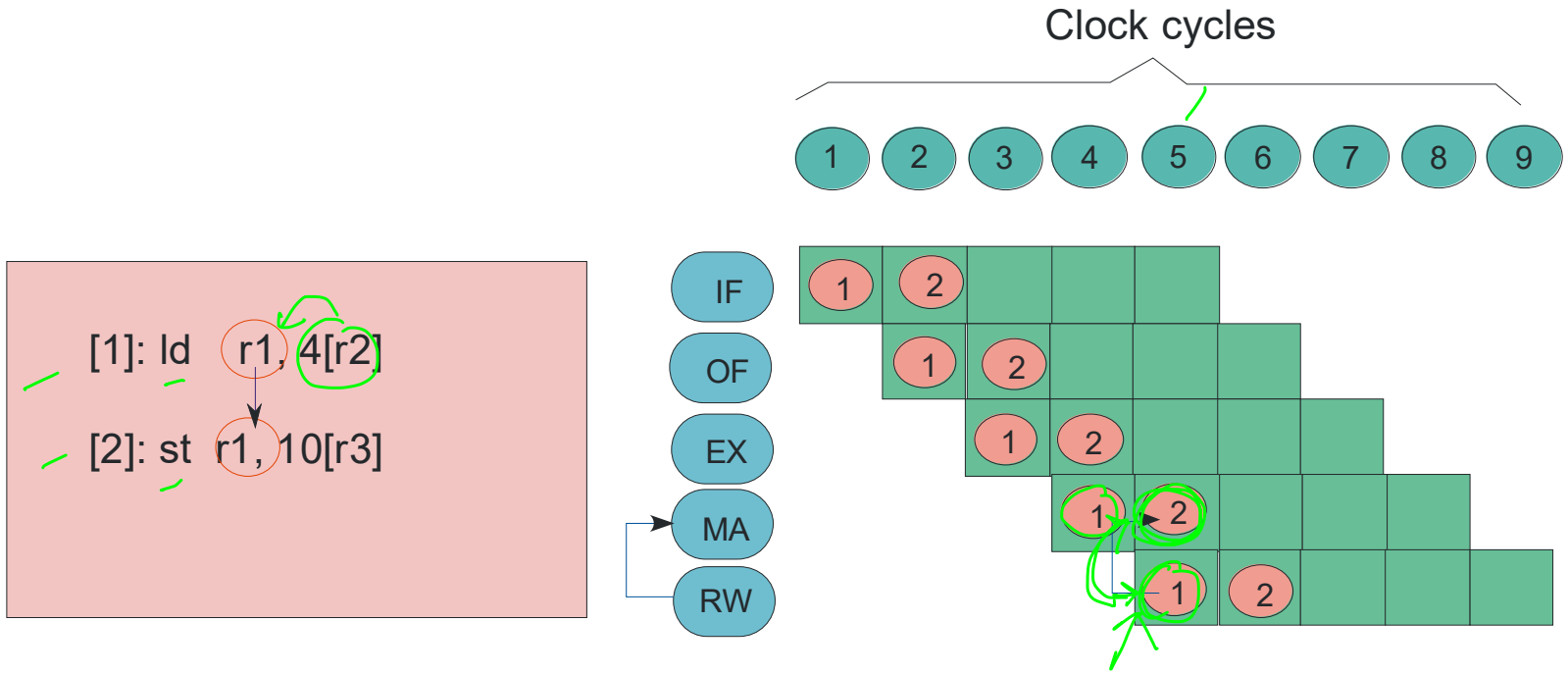
* We need to add a multitude of forwarding paths
* Rules for creating forwarding paths
  * Add a path from a later stage to an earlier stage
  * Try to add a forwarding path as late as possible. For, example, we avoid the EX → OF forwarding path, since we have the MA → EX forwarding path
  * The IF stage is not a part of any forwarding path.

Forwarding Path

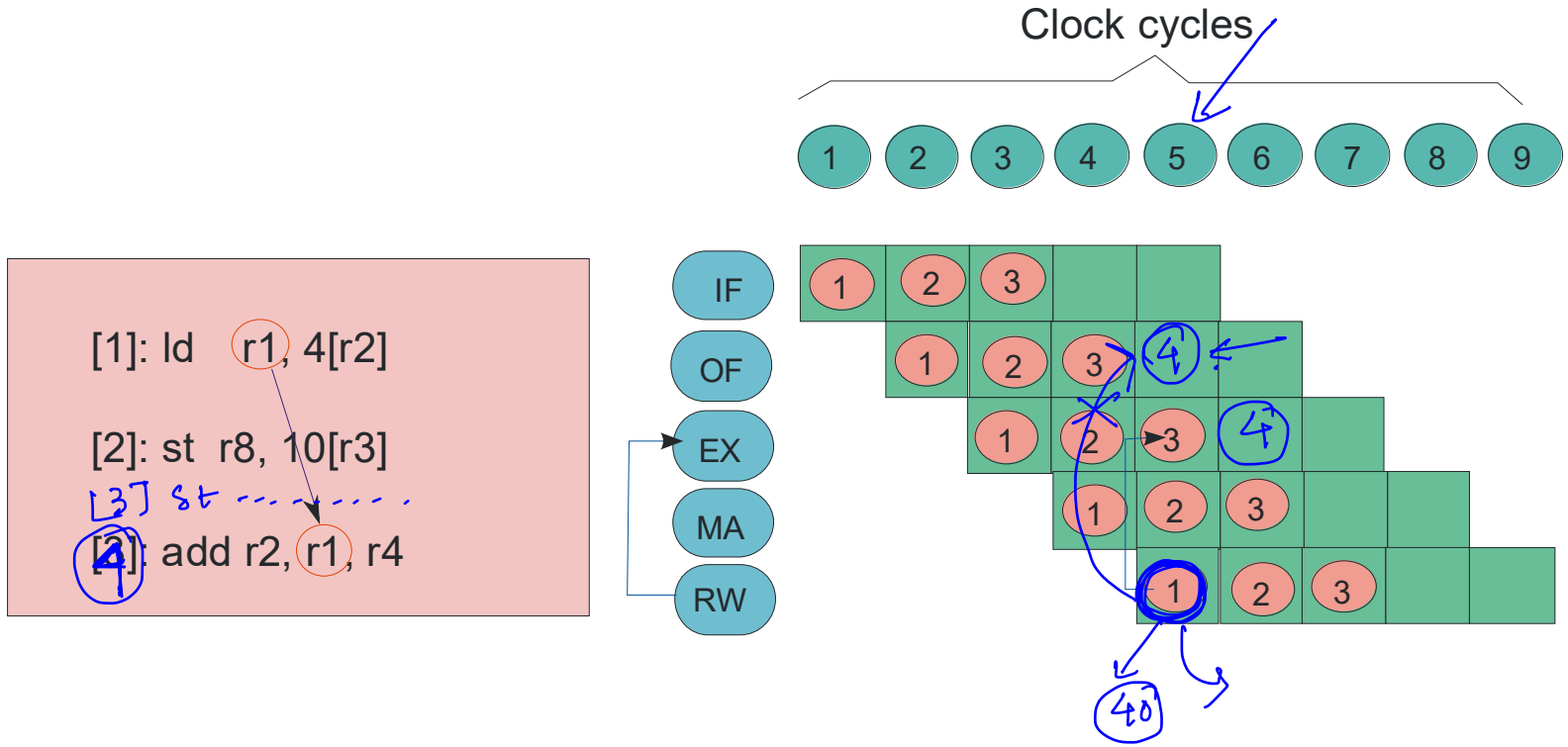
- 3 Stage Paths
  - RW → OF
- 2 Stage Paths
  - RW → EX
  - MA → OF (X Not Required)
- 1 Stage Paths
  - RW → MA (load to store)
  - MA → EX (ALU Instructions, load, store)
  - EX → OF (X Not Required)



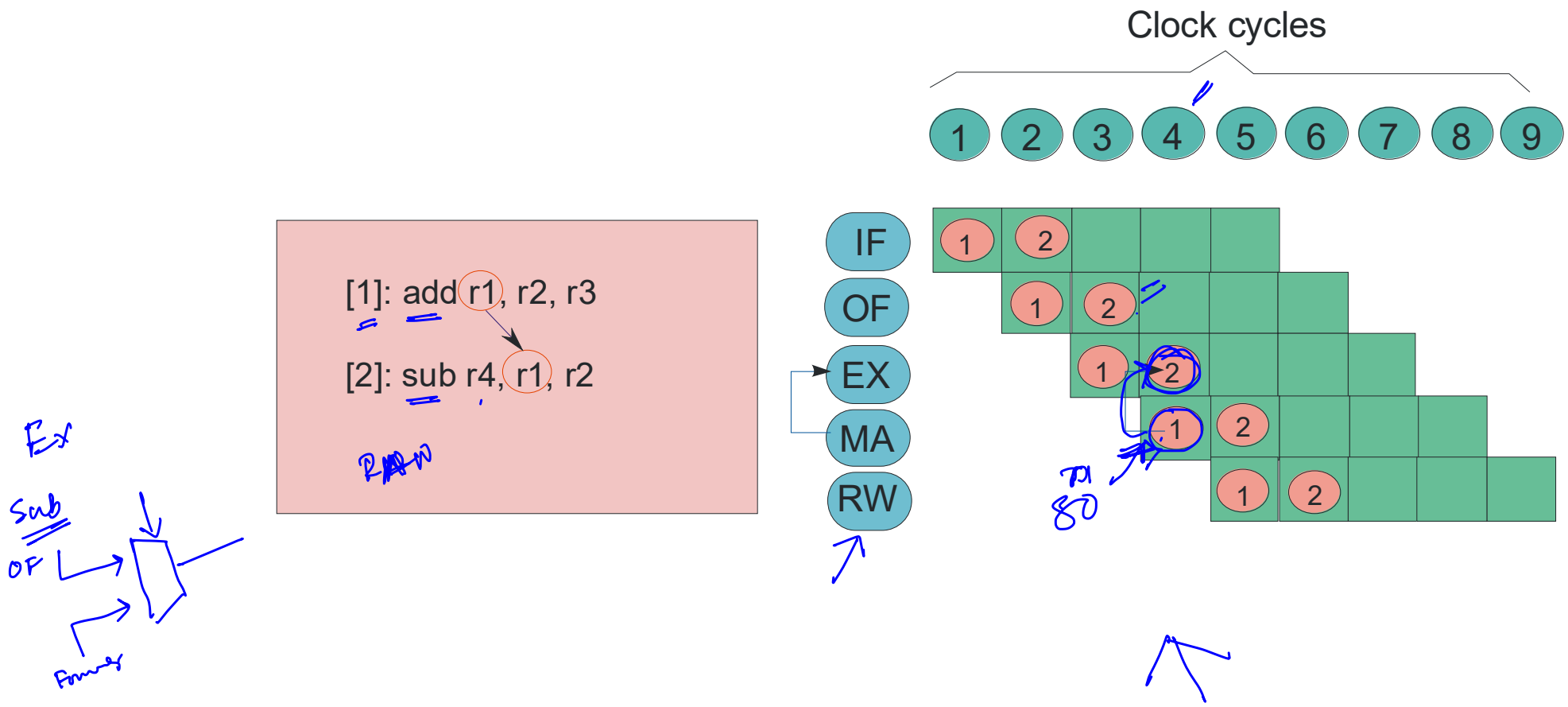
Forwarding Paths : RW → MA



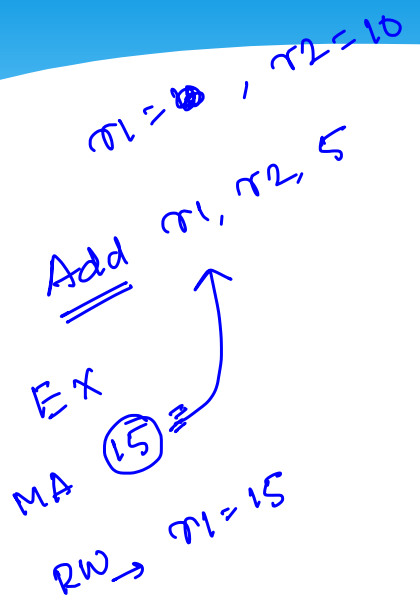
Forwarding Paths : RW → EX

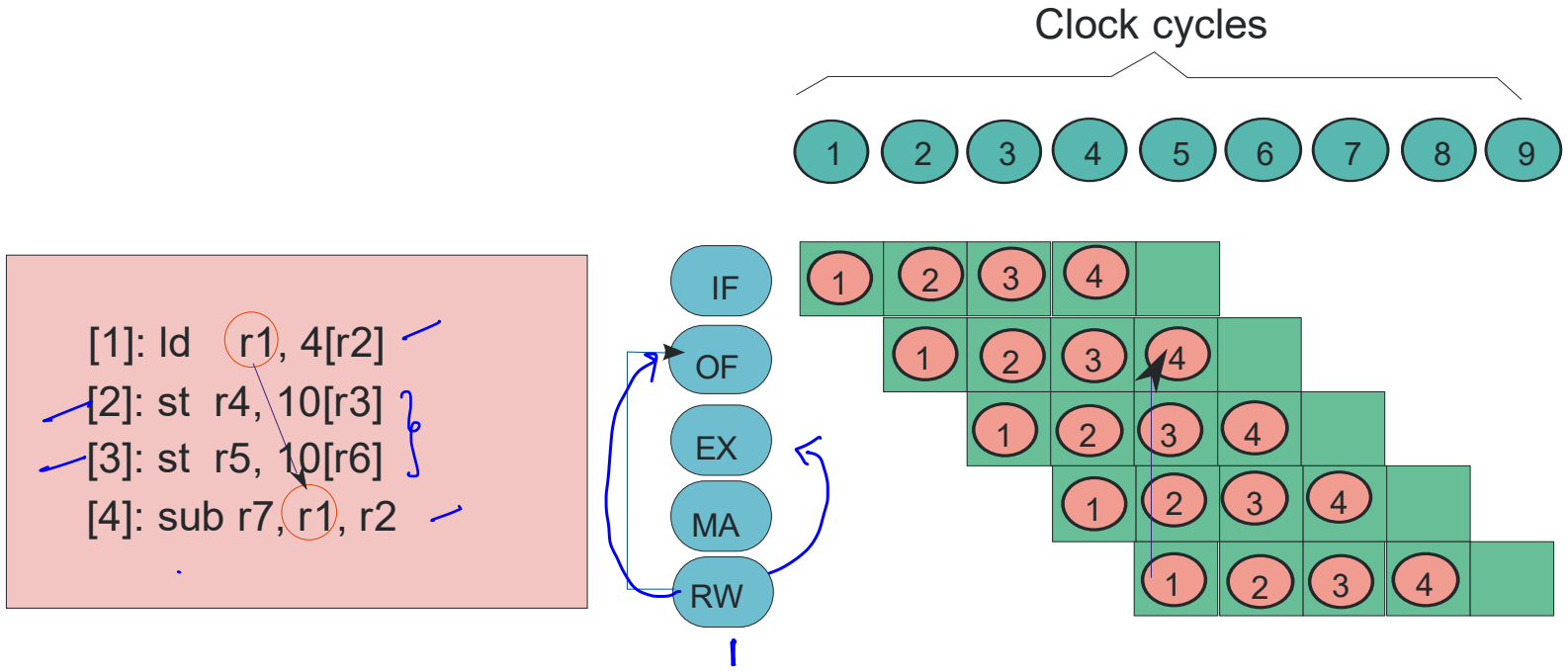


Forwarding Path : MA → EX



Forwarding Path : RW → OF

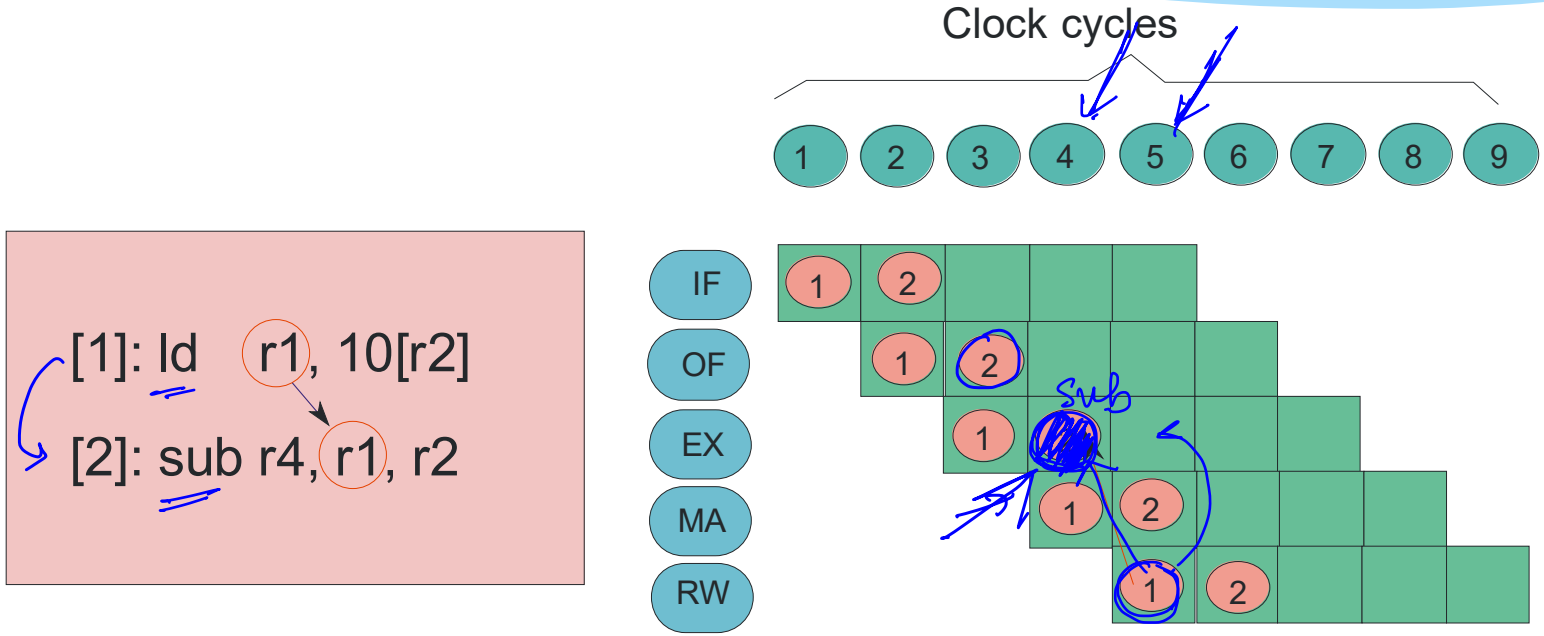




Data Hazards with Forwarding

* Forwarding has unfortunately not eliminated all data hazards
* We are left with one special case.
* Load-use hazard
  * The instruction immediately after a load instruction has a RAW dependence with it.

Load-Use Hazard

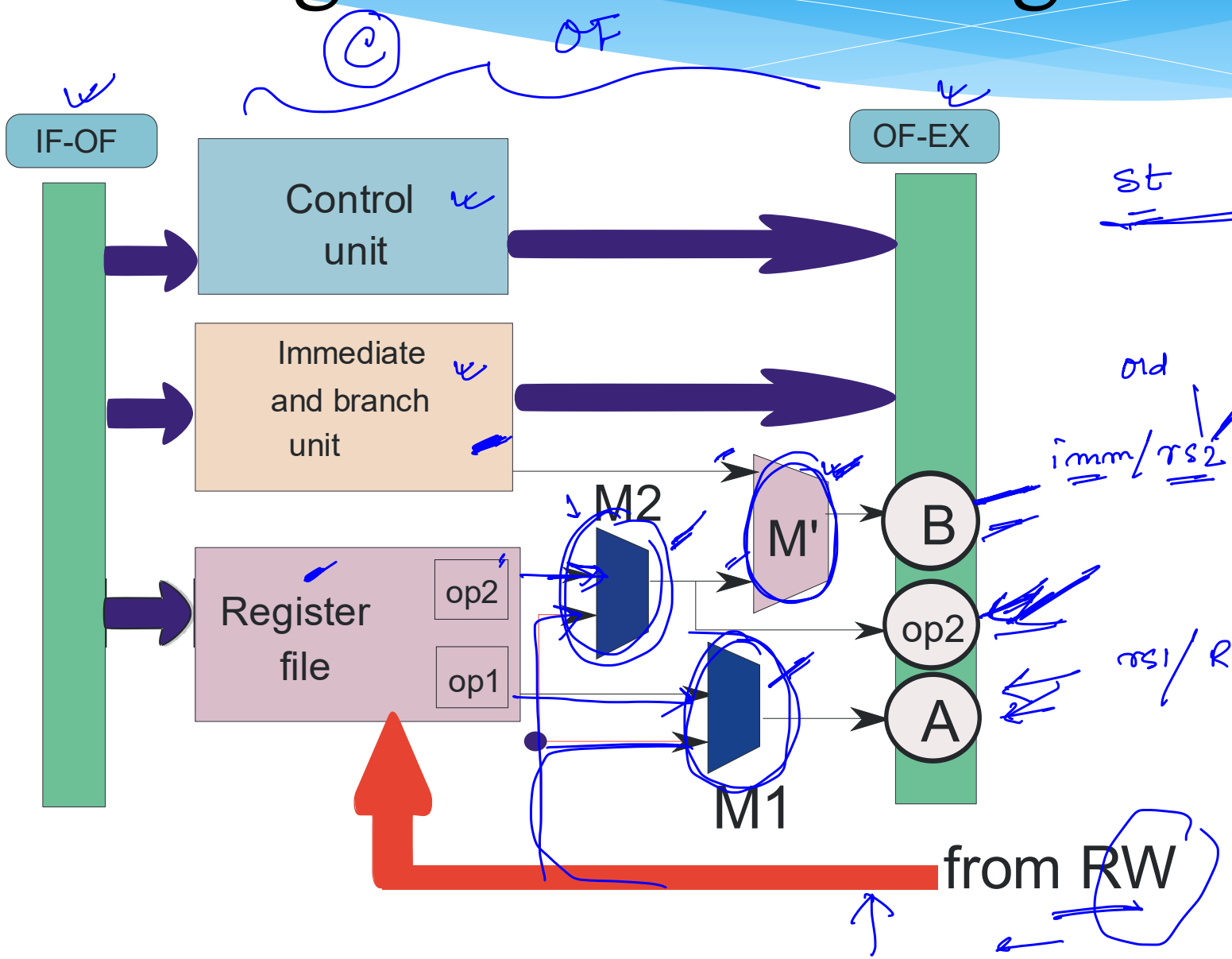


* Cannot forward (arrow goes backwards in time)
* Need to add a bubble (then use RW → EX forwarding)

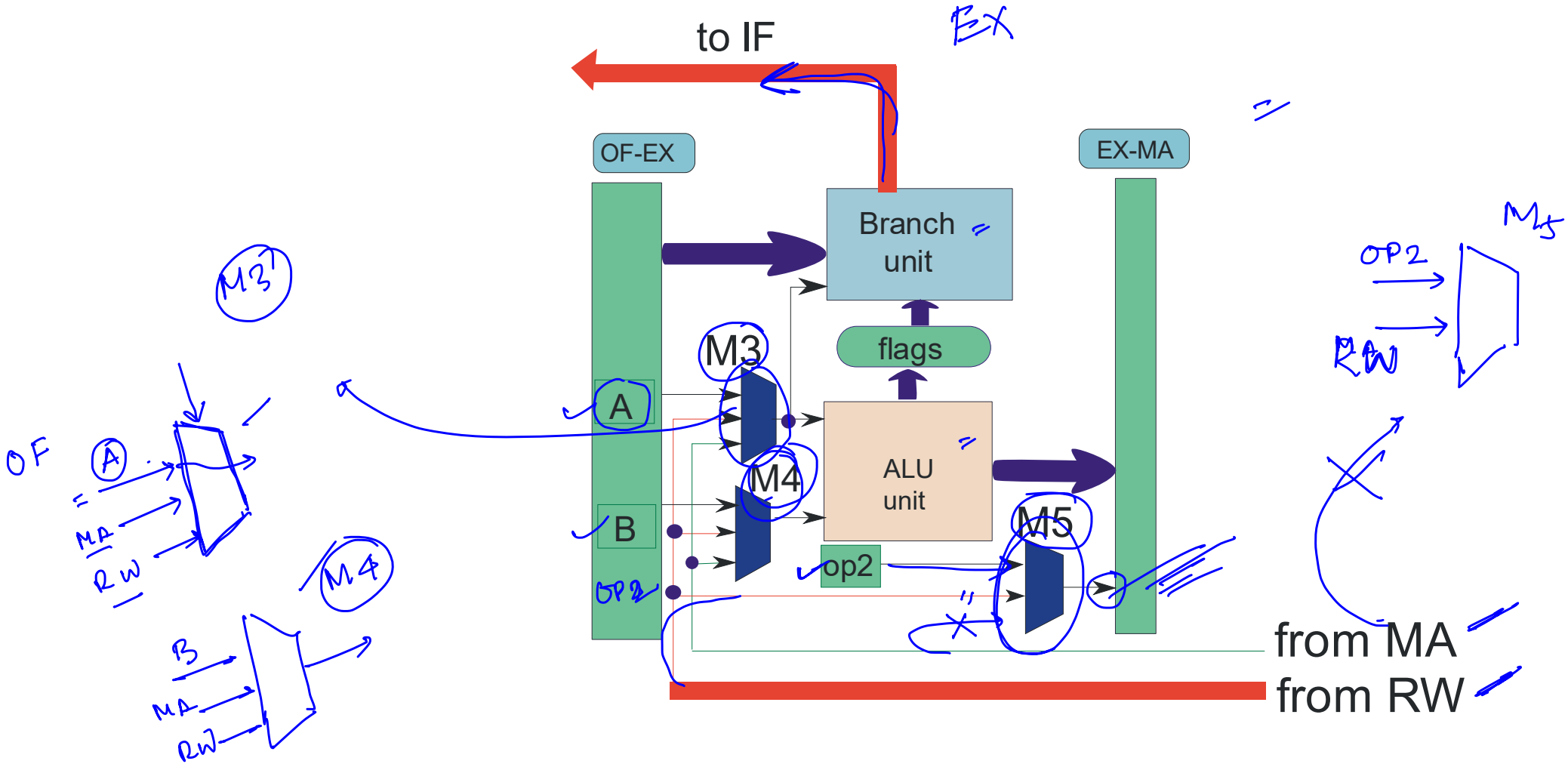
Implementation of Forwarding

* At every stage there is a choice of inputs for each functional unit
  * Use the default inputs from the previous stage
  * OR, use one of the forwarded inputs
  * Use a multiplexer to choose between the inputs
  * A dedicated forwarding unit generates the signals for the **forwarding multiplexers**

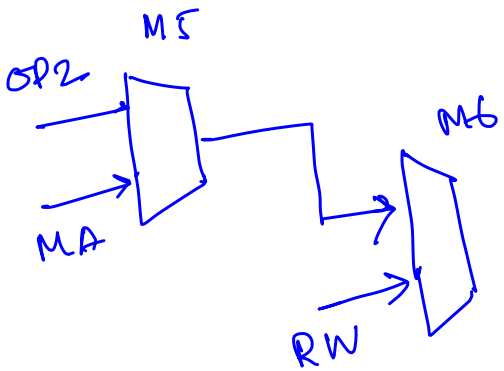
OF Stage with Forwarding

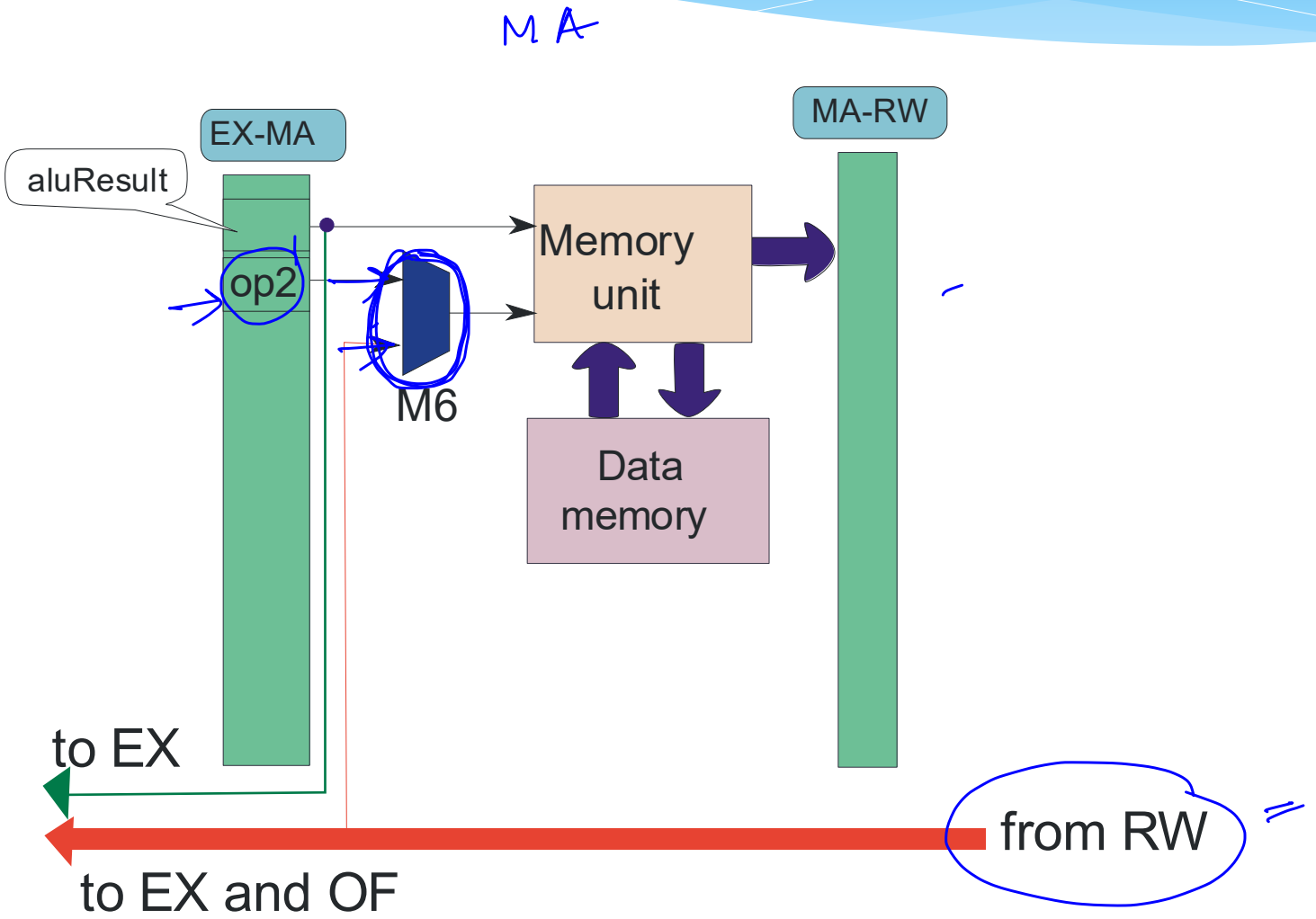


EX Stage with Forwarding

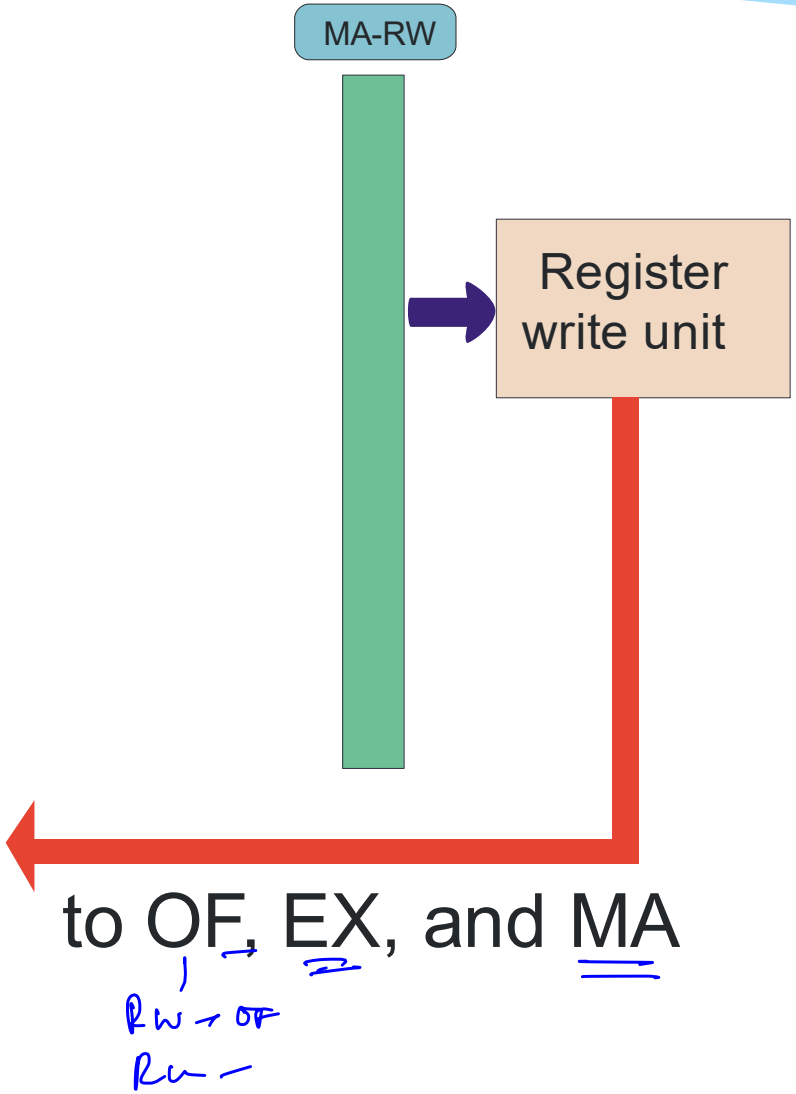


MA Stage with Forwarding

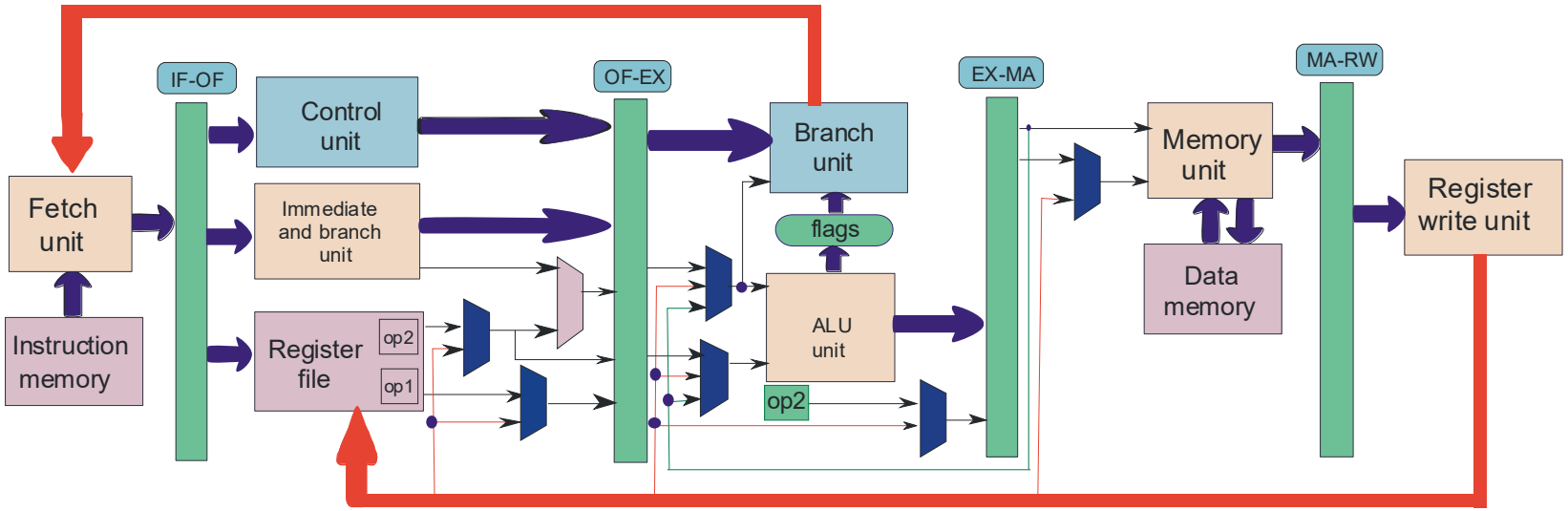




RW Stage with Forwarding



Data Path with Forwarding



Forwarding Conditions

* Determine if there is a conflict between two instructions in different stages
  * Find if there is a conflict for the first operand (rs1/ ra)
  * Find if there is a conflict for the second operand (rs2/rd)
* Always forward from the **latest** instruction (that is earlier than the current instruction)

Algorithm 6: Conflict on the first operand (rs1/ra)

Data: instructions, [A], and [B] (possible forwarding: [B] → [A])
Result: conflict exists on rs1/ra (**true**), no conflict (**false**)
if *[A].opcode ∈(nop,b,beq,bgt,call,not,mov)* **then**
    /* Does not read from any register */
    return false
end
if *[B].opcode ∈(nop, cmp, st, b, beq, bgt, ret)* **then**
    /* Does not write to any register */
    return false
end
/* Set the sources */
src1 ← *[A].rs1*
if *[A].opcode = ret* **then**
    src1 ← *ra*
end
/* Set the destination */
dest ← *[B].rd*
if *[B].opcode = call* **then**
    dest ← *ra*
end
/* Detect conflicts */
if *src1 = dest* **then**
    return true
end
return false

Algorithm 7: Conflict on the second operand (rs2/rd)
Data: instructions, [A], and [B] (possible forwarding: [B] → [A])
Result: conflict exists on second operand (rs2/rd) (**true**), no conflict (**false**)

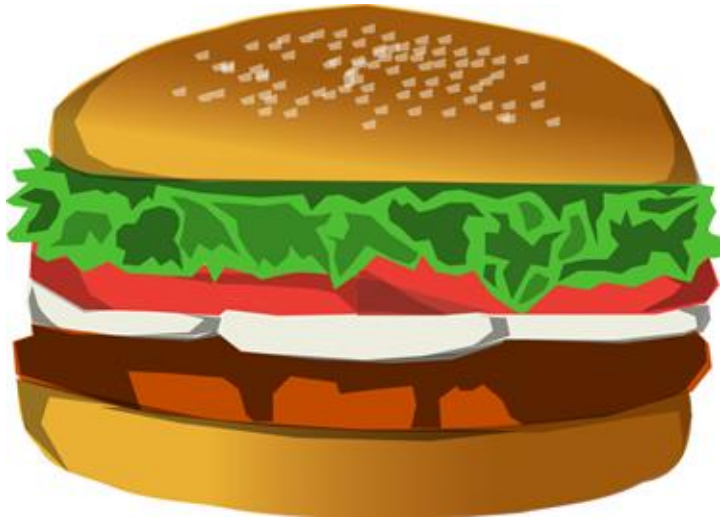
```
if [A].opcode ∈ (nop,b,beq,bgt,call) then
    /* Does not read from any register */
    return false
end
if [B].opcode ∈ (nop, cmp, st, b, beq, bgt, ret) then
    /* Does not write to any register */
    return false
end
/* Check the second operand to see if it is a register */
if [A].opcode ≠( st) then
    if [A].I = 1 then
        return false
    end
end
/* Set the sources */
src2 ← [A].rs2
if [A].opcode = st then
    src2 ← [A].rd
end
```

```
/* Set the destination */
dest ← [B].rd
if [B].opcode = call then
    dest ← ra
end
/* Detect conflicts */
if src2 = dest then
    return true
end
return false
```

Interlocks with Forwarding

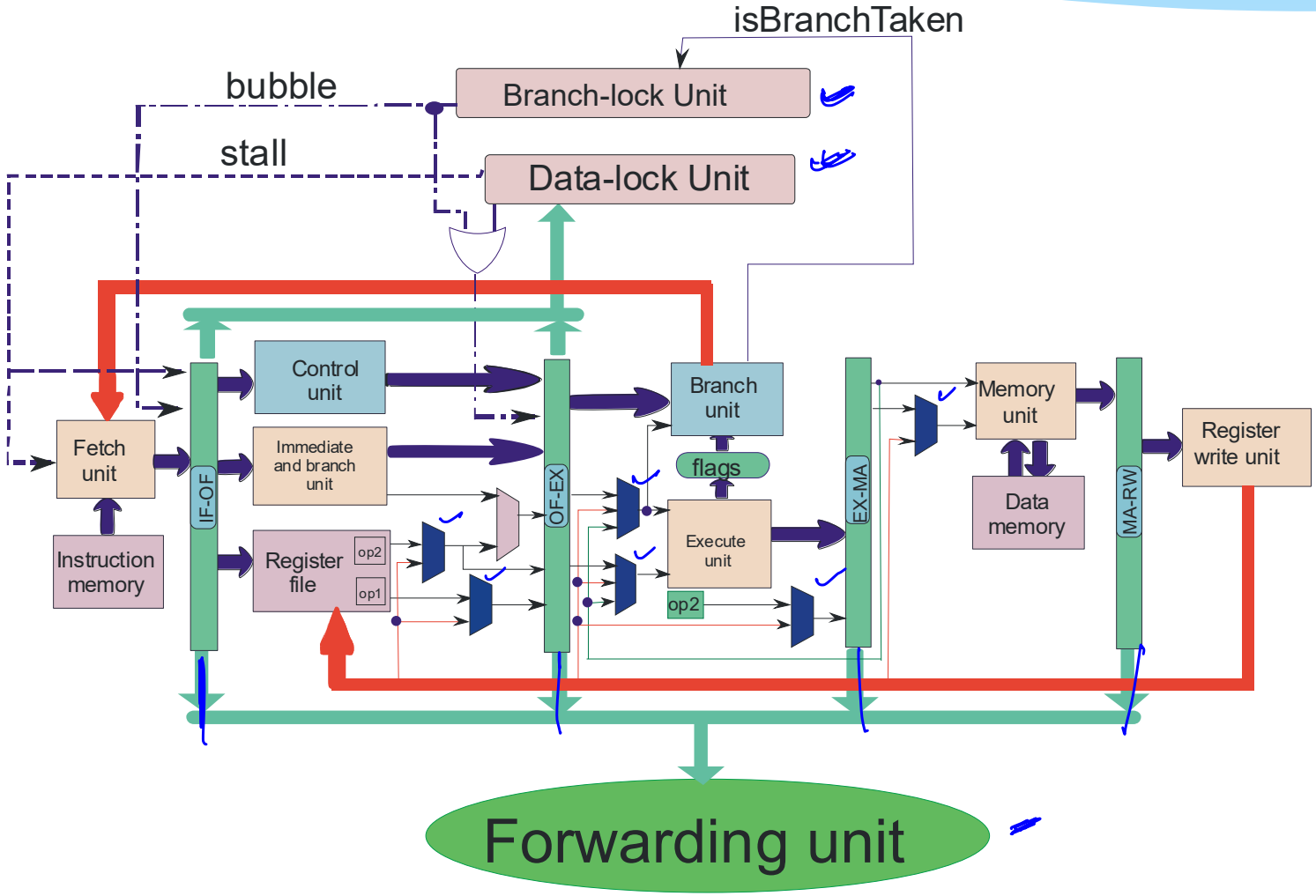
* Data-Lock
  * We need to only check for the load-use hazard
  * If the instruction in the EX stage is a load, and the instruction in the OF stage uses its loaded value, then stall for 1 cycle
* Branch-Lock
  * Remains the same as before.

The Curious Case of the *call* instruction



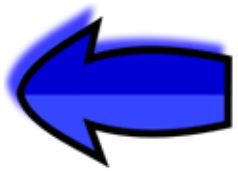
* The call instruction generates the value of ra in the RW stage
* Any instruction that uses the value of ra (such as ret) still works **correctly** !!!
* Prove it ....

Complete Data Path



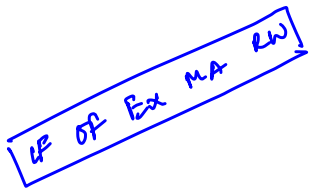
Outline

* Overview of Pipelining
* A Pipelined Data Path
* Pipeline Hazards
* Pipeline with Interlocks
* Forwarding
* Performance Metrics
* ~~Interrupts/ Exceptions~~



Measuring Performance

* What do we mean by the performance of a processor ?
  * **ANSWER** : Almost nothing
* What should we ask instead ?
  * What is the performance with respect to a given program or a set of programs ?
  * Performance is inversely proportional to the time it takes to execute a **program**



Performance $\propto \frac{1}{T}$ = # seconds.

$$= \frac{\#\text{ seconds}}{\#\text{ cycles}} \times \frac{\#\text{ cycles}}{\#\text{ instructions}} \times \#\text{ instructions}$$

: no. of

f : freq. = cycles per second

CPI : Cycles per instruction

$$T = \frac{1}{f} \times CPI \times \#\text{ instrn}$$

$$\text{Performance} = P = \frac{1}{T} = \frac{f}{CPI \times \#\text{ instrn}} = \frac{f \times IPC}{\#\text{ instrn}}$$

$$IPC = \frac{1}{CPI} = \text{Instrn per cycle}$$

$$P = \frac{f \times IPC}{\#\text{ instrn}}$$

$\propto$

for ( ) 5

{ ≡ } 10

}

Static instrn

Dynamic instrn ← 50

Computing the Time a Program Takes

$$\tau = \#seconds$$
$$= \frac{\#seconds}{\#cycles} * \frac{\#cycles}{\#instructions} * (\#instructions)$$
$$= \underbrace{\frac{\#seconds}{\#cycles}}_{1/f} + \underbrace{\frac{\#cycles}{\#instructions}}_{CPI} * (\#instructions)$$
$$= \frac{CPI * \#insts}{f}$$

* CPI → Cycles per instruction
* f → frequency (cycles per second)

The Performance Equation

$$P \propto \frac{IPC * f}{\#insts}$$

* IPC → 1/CPI (Instructions per Cycle)
* What are the units of performance ?
  * **ANSWER** : arbitrary

Number of Instructions (#insts)

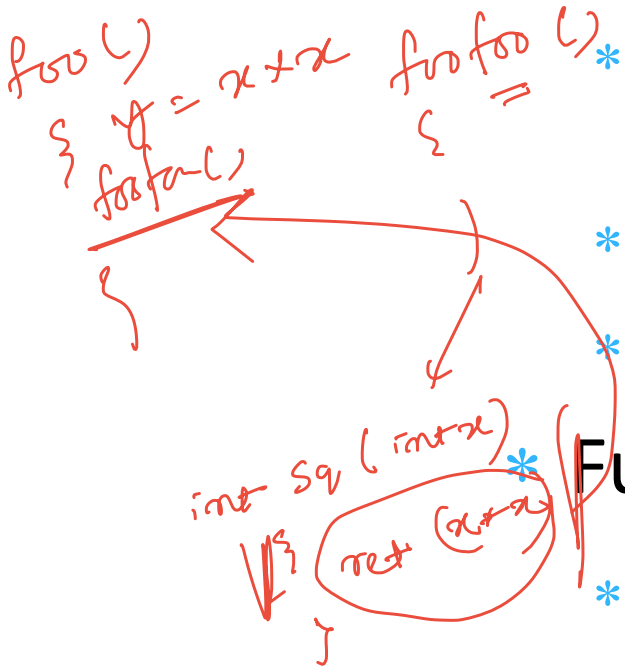
Static Instruction: The binary or executable of a program, contains a list of ***static instructions***.

Dynamic Instruction: A **dynamic instruction** is a running instance of a static instruction, which is created by the processor when an instruction enters the pipeline.

* Note that these are dynamic instructions
  * **NOT** static instructions
* A smart compiler can reduce the number of executed instructions

Number of Instructions(#insts) – 2

* Dead code removal



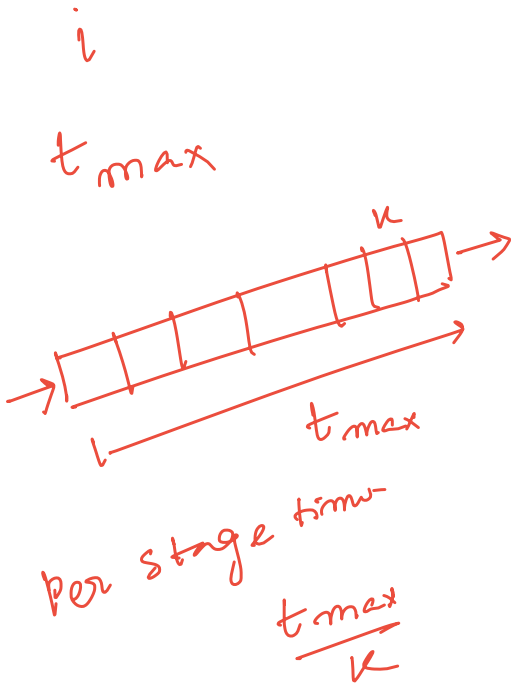
  * Often programmers write code that does not determine the final output
  * This code is redundant
  * It can be identified and removed by the **compiler**
* Function inlining
  * Very small functions have a lot of overhead → call, ret instructions, register spilling, and restoring
  * Paste the code of the callee in the code of the caller (known as **inlining**)

Computing the CPI

- CPI for a single cycle processor = 1
- CPI for an ideal pipeline(no hazards)
  - Assume we have *n* instructions, and *k* stages
  - The first instruction enters the pipeline in cycle 1
  - It leaves the pipeline in cycle k
  - The rest of the **(n-1) instructions** leave in the next (n-1) consecutive cycles

$$CPI = \frac{n + k - 1}{n}$$

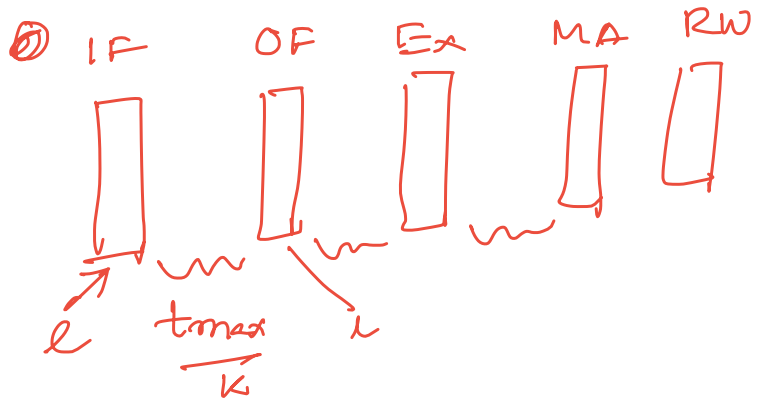
Computing the Maximum Frequency



- Let the maximum amount of time that it takes to execute any instruction be :
  - **t_{max}** (also known as algorithmic work)
- Minimum clock cycle time of a single cycle pipeline → t_{max}
- In the case of a pipeline, let us assume that all the pipeline stages are **balanced**
- Time per stage → t_{max} / k

Maximum Frequency - II

* Let the **latch delay** be **l**
* We thus have :



$$\frac{1}{f} = t_{stage} = \frac{t_{max}}{k} + l$$

$$\frac{1}{f} = \frac{t_{max}}{k} + l$$

The minimum cycle time (1/f) is equal to t_{stage}. Let us thus, assume that our cycle time is as low as possible.

$P \propto f \times IPC$

$\frac{f}{CPI}$

$= \frac{1}{\frac{t_{max}}{k} + l}$

$= \frac{n}{\left(\frac{t_{max}}{k} + l\right) \times (n + k - 1)}$

$P = \frac{n}{(n-1)\frac{t_{max}}{k} + \left(\frac{t_{max}}{k} + ln - l\right) + lk}$

$\frac{\partial P}{\partial k} = 0$

Performance of an Ideal Pipeline

* Let us assume that the number of instructions are a **constant**

$$P = \frac{f}{CPI}$$

$$= \frac{1}{\frac{\frac{t_{max}}{k} + l}{\frac{n + k - 1}{n}}}$$

$$= \frac{n}{\left(\frac{t_{max}}{k} + l\right) * (n + k - 1)}$$

$$= \frac{n}{\frac{(n-1)t_{max}}{k} + (t_{max} + \ln - l) + lk}$$

Optimal Number of Pipeline Stages

$$\frac{\partial\left(\frac{(n-1)t_{max}}{k}+(t_{max}+\ln-l)+lk\right)}{\partial k}=0$$

$$\Rightarrow -\frac{(n-1)t_{max}}{k^2}+l=0$$

$$\Rightarrow k=\sqrt{\frac{(n-1)t_{max}}{l}}$$

$t_{max}\uparrow$ $k\uparrow$

$n\to\infty$ $k\to\infty$

* k is inversely proportional to $\sqrt{l}$
* k is proportional to $\sqrt{t_{max}}$

Implications

* As we increase the latch delay, we should have less pipeline stages
  * We need to minimise the time wasted in accessing latches
* As we increase the amount of **algorithmic work**, we require more pipeline stages for ideal performance
  * More pipeline stages help distribute the work better, and increase the overlap across instructions

Implications - II

* As the number of instructions tends to ∞, the number of ideal pipeline stages also tends to ∞
  * The higher startup time gets amortized in the long run

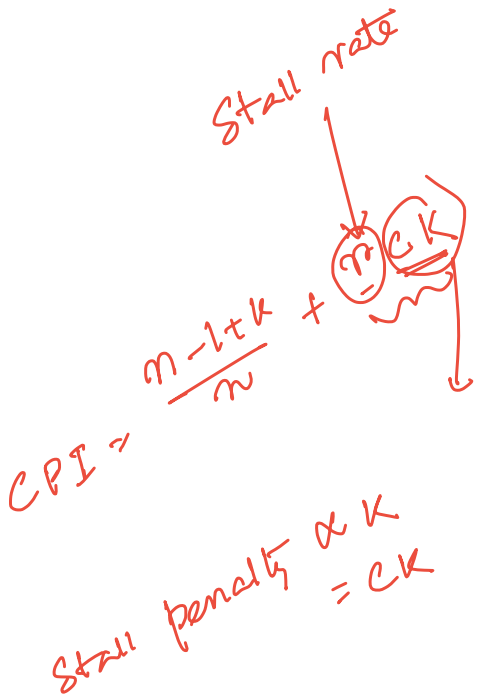
A Non-Ideal Pipeline

* Our ideal CPI (CPI_{ideal} = 1) is 1
* However, in reality, we have stalls

$$CPI = CPI_{ideal} + stall\_rate * stall\_penalty$$

* Let us assume that the stall rate is a function of the program, and its nature of dependences

Non-Ideal Pipeline - II



* Let us assume that the stall penalty is proportional to the number of pipeline stages
* Both these assumptions are strictly not correct. They are being used to make a coarse grained mathematical model.
* CPI = (n+k-1)/n + rck
  * r → stall rate, c → constant of proportionality

$$P \propto \frac{f}{CPI}$$

$$\frac{1}{\frac{t_{max}+1}{K} + \frac{n-1+K}{n} + rcK}$$

$\downarrow K \propto \frac{1}{r} \uparrow$

c

$$= \frac{\partial P}{\partial K} \to 0 \Rightarrow K = \sqrt{\frac{(n-1)\,t_{max}}{\ell\,(1+rcn)}}$$

$$K = \sqrt{\frac{t_{max}}{\ell rc}}$$

$n \to \infty$

$\ell << n$

Mathematical Model

$$P = \frac{f}{CPI}$$

$$= \frac{\frac{1}{\frac{t_{max}}{k} + l}}{\frac{n + k - 1}{n} + rck}$$

$$= \frac{n}{\frac{(n-1)t_{max}}{k} + (rcnt_{max} + t_{max} + \ln - l) + lk(1 + rcn)}$$

Mathematical Model - II

$$\frac{\partial\left(\frac{(n-1)t_{max}}{k}+(rcnt_{max}+\ t_{max}+\ln-l)+lk(1+rcn)\right)}{\partial k}=0$$

$$\Longrightarrow -\frac{(n-1)t_{max}}{k^2}+l(1+rcn)=0$$

$$\Longrightarrow k=\sqrt{\frac{(n-1)t_{max}}{l(1+rcn)}}\approx\sqrt{\frac{t_{max}}{lrc}}\quad(as\ n\longrightarrow\infty)$$

Implications

* For programs with a lot of dependences (high value of r) → Use less pipeline stages
* For a pipeline with forwarding → c is smaller (than a **pipeline** that just has interlocks)
  * It requires a larger number of pipeline stages for optimal performance

Implications

* The optimal number of pipeline stages is directly proportional to $\sqrt{(t_{max} / l)}$
  * This ratio is not significantly changing across technologies.
  * This explains why the number of pipline stages has remained more or less constant for the last 5-10 years

Example

Example Consider two programs that have the following characteristics.

Program 1		Program 2	
Instruction Type	Fraction	Instruction Type	Fraction
loads	0.4	loads	0.3
Branches	0.2	Branches	0.1
ratio(taken branches)	0.5	ratio(taken branches)	0.4

The ideal CPI is 1 for both the programs. Let 50% of the load instructions suffer from a load use hazard. Assume that the frequency of P_1 is 1, and the frequency of P_2 is 1.5. Here, the units of the frequency are not relevant. Compare the performance of P_1 and P_2.

$P = \frac{f}{CPI}$

P_1

$CPI = CPI_{ideal} + \sum (\text{stall rate} \times \text{stall penalty})$

$= 1 + (0.4 \times .5 \times 1 + 0.2 \times .5 \times 2) = 1.4$

P_2

$CPI = 1 + (0.3 \times .5 \times 1) + (0.1 \times 0.4 \times 2) = 1.23$

$P_1 = f/CPI = 1/1.4 = 0.71$

$P_2 = 1.5/1.23 = 1.22$